

水泥烧成系统电除尘器的协同优化控制

摘 要

针对水泥烧成系统四电场电除尘器的节能与超低排放协同控制问题,本文审计7天、10080条分钟级记录。出口浓度有效值全部位于 $48.74\sim 50.00\text{ mg/Nm}^3$,且未覆盖10或 5 mg/Nm^3 目标区间,故附件只能支持工况和总功率辨识,不能直接识别目标附近的控制—排放响应。本文构建“**工况聚类—数据驱动功率—工程灰箱排放—历史支持域优化**”框架,以滚动验证选择二次岭功率模型,以Deutsch-Anderson关系、积灰损失和振打再飞扬代理构造排放情景,并采用5个随机种子和局部细化搜索可行解。功率模型验证集 R^2 为 **0.9837**、RMSE 为 **7.12 kW**,回顾性测试 R^2 为 0.9571。在出口记录按0.1缩放、采用中心灰箱参数且控制量位于历史支持域的条件下,10和 5 mg/Nm^3 对应的工况加权功率分别为1609.4和1829.8 kW, **条件式功率增幅约 13.7%**。405组灰箱参数组合情景均可行,功率增幅为 **11.37%~14.31%**;场权重消融后,前两电场承担 **70.8%~73.7%** 的正向电压增量。所有排放数值均为条件式推演,不作为附件直接识别的实证结论。

关键词: 电除尘器; 工况聚类; 岭回归; 灰箱模型; 振打再飞扬; 约束优化

1 问题重述与分析

1.1 问题背景

电除尘器通过高压电场使粉尘荷电并向收尘极迁移。提高电场电压通常能够增强荷电与迁移,但会提高运行功率并受到火花、绝缘和联锁条件限制;振打过于频繁会增加再飞扬次数,周期过长又可能导致积灰加重和单次脱落量增大。水泥工业颗粒物排放受到国家标准约束 [1], 题目进一步给出 10 mg/Nm^3 和 5 mg/Nm^3 两个控制目标, 要求同时考虑排放、总功率和振打峰值。

本文研究对象的四电场电除尘器结构及其可测量与可调量如图 1 所示: 含尘烟气依次流经四个电场, 每个电场由独立高压电源施加电压 U_i 并由独立振打装置按周期 T_i 清灰, 入口侧可测入口温度 H 、入口粉尘浓度 C_{in} 与烟气流量 Q , 出口侧可测出口粉尘浓度 C_{out} 与总功率 P 。

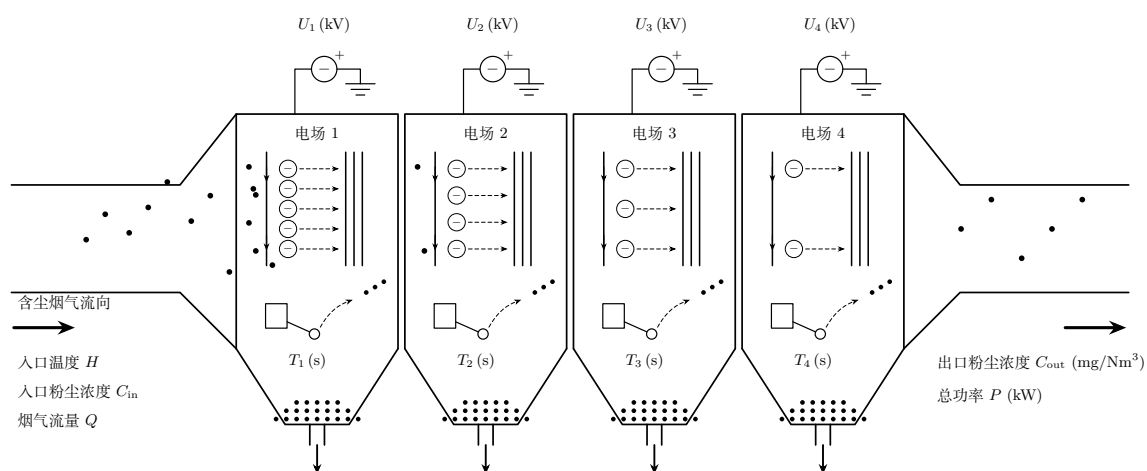


图 1 四电场电除尘器结构与变量定义

题目包含四项相互依赖的任务: 第一, 解释入口温度、入口浓度、烟气流量、四电场电压和振打周期与出口浓度之间的关系, 并分析振打峰值; 第二, 划分典型工况, 在 10 mg/Nm^3 约束下寻找最低功率控制参数; 第三, 选取差异显著的工况解释控制优先级; 第四, 将约束收紧至 5 mg/Nm^3 并估计功率代价。后面三问都以第一问的排放响应为基础, 因此必须先判断附件能否识别该响应。

1.2 总体思路

若直接把出口浓度作为监督学习目标, 模型可能只是在复现传感器封顶值, 而不是学习除尘过程。本文采用“先识别、后优化”的路线:

1. 审计时间连续性、缺失、量纲和出口浓度支持域;
2. 仅利用入口温度、入口浓度和流量划分工况, 避免用结果变量定义工况;
3. 对可由数据验证的总功率建立岭回归模型;
4. 对不可由附件辨识的排放层引入明确标注的工程情景先验;
5. 在历史支持域内搜索控制量, 并通过收敛性与先验敏感性评价结果。

该路线的核心不是把假设包装成数据结论, 而是把“实测证据”与“条件式推演”分层, 使数值答案能够被复算、被质疑, 也能够获得新数据后替换。

2 数据预处理与可识别性门禁

2.1 数据完整性与派生变量

附件覆盖 2024 年 5 月 1 日 0 时至 5 月 7 日 23 时 59 分，共 10080 条记录，采样间隔为 1 min；时间戳无重复、无缺口。原始字段包括入口温度 H_t 、入口浓度 $C_{in,t}$ 、烟气流量 Q_t 、四电场电压 $U_{i,t}$ 、四电场振打周期 $T_{i,t}$ 、出口浓度 $C_{out,t}$ 和总功率 P_t 。出口浓度缺失 50 条，缺失比例 0.50%。

为表达过程负荷和振打风险，构造粉尘质量负荷

$$L_t = C_{in,t} Q_t / 1000 \quad (1)$$

其中 C_{in} 单位为 g/Nm^3 ， Q 单位为 Nm^3/h ，故 L 单位为 kg/h 。另构造电压平方和、前后级电压梯度、振打频率 $60/T_i$ 以及每周期积尘代理 $L_i T_i / 3600$ 。所有聚类与回归参数仅在训练期估计，验证期和测试期不参与拟合。

出口浓度分布及截顶情况如图 2 所示。有效记录全部位于 $48.74 \sim 50.00 \text{ mg}/\text{Nm}^3$ ，5491 条记录恰好等于 $50 \text{ mg}/\text{Nm}^3$ ，数据没有覆盖题设控制目标区间。

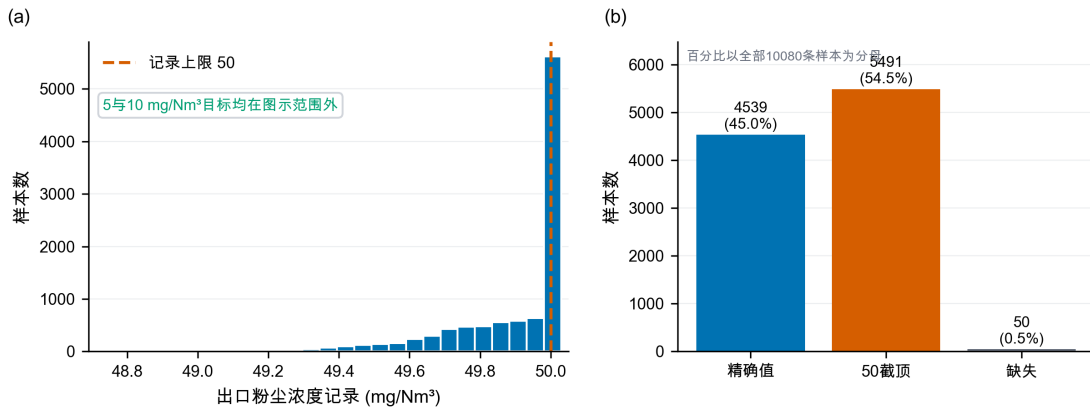


图 2 出口浓度数据诊断

2.2 出口浓度可识别性审计

有效出口浓度共 10030 条，范围仅为 $48.74 \sim 50.00 \text{ mg}/\text{Nm}^3$ ，中位数等于 $50 \text{ mg}/\text{Nm}^3$ ，其中 5491 条恰好位于 50。更关键的是，没有任何样本不超过 $10 \text{ mg}/\text{Nm}^3$ 或 $5 \text{ mg}/\text{Nm}^3$ 。同期 Pearson 和 Spearman 相关系数以及 1、5、15、30、60、120 min 滞后检查均未发现绝对值超过约 0.015 的稳定关系；已有时间验证结果还表明，封顶分类 AUC 仅为 0.5098，未封顶样本回归验证 $R^2 = -0.0190$ ，记录值回归验证 $R^2 = -0.0219$ 。这些结果只能说明“当前记录无法提供可学习信号”，不能反推物理变量对真实排放毫无影响。已有数据驱动研究使用宽负荷下的机组负荷与各电场二次电流训练出口浓度模型 [8]；对照来看，本附件既缺少二次电流，出口记录也没有足够变化，故不具备相同的辨识条件。

首 24 h 过程量的变化如图 3 所示。入口变量和功率具有连续变化和工况迁移特征，而出口浓度记录几乎被压缩在单点附近，进一步说明两类数据的信息量不对称。

2.3 小数点情景及边界

为继续完成题目的数值部分，本文接受如下竞赛情景：真实出口浓度可能为记录值的 0.1 倍，即历史锚点约为 $4.874 \sim 5.000 \text{ mg}/\text{Nm}^3$ 。该假设能够解释“50 附近封顶”，但附件本身无法证明其正确。即使接受该假设， $10 \text{ mg}/\text{Nm}^3$ 目标也在历史样本中始终宽松满足，而 $5 \text{ mg}/\text{Nm}^3$ 附近仍有大量截顶观

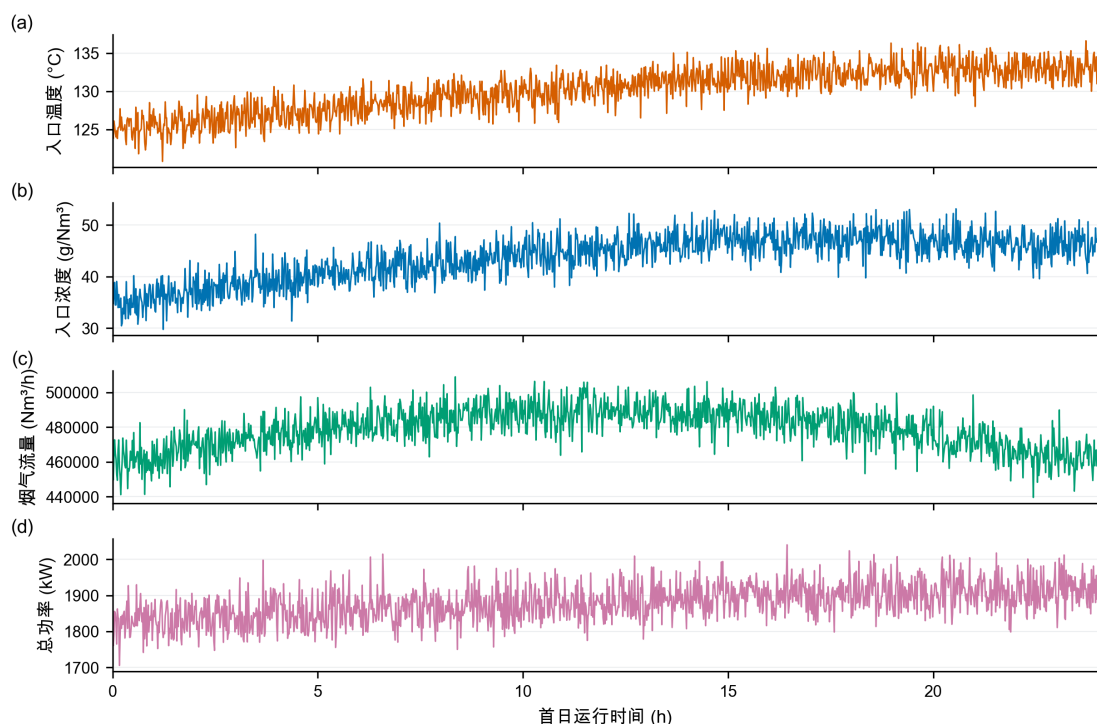


图3 首24 h 过程量时序

测，因此 10 到 5 mg/Nm^3 的反事实功率增幅仍不能从数据直接回归，只能由灰箱先验推演。后文所有排放值均以“情景值”表述。

3 模型假设、符号与总体框架

3.1 模型假设

本文将假设分为数据事实、情景假设、优化边界与运行约束四类，并逐条标注其证据状态，如表 1 所示。

表 1 模型假设及证据状态

类别	假设	证据状态
数据事实	7 天记录按分钟连续，除 50 条出口浓度缺失外不填造目标值	附件可核验
数据事实	功率关系按时间滚动验证，不随机打乱	可由验证结果
情景假设	中心情景将出口记录整体乘 0.10；结构扫描取 0.08~0.12	附件不可确认
情景假设	电场贡献比较等权、弱前级与中心前级三种形状	工程先验与消
情景假设	振打峰值指数取 1.10、1.35、1.60，相位叠加尺度取 0.65、0.80、1.00	工程结构情景
优化边界	控制量位于工况训练期 P05~P95，且 Mahalanobis 距离不超过历史经验 97.5% 分位数	历史支持域，
运行约束	前级电压高于后级至少 3 kV，后级振打周期长于前级至少 100 s	工程排序约束

3.2 主要符号

全文主要符号及其单位如表 2 所示。

本文采用的数据驱动层与灰箱情景层协同框架如图 4 所示。工况和总功率由实测数据辨识，排放约束由显式先验建立，两者仅在设计域优化阶段合并。

表 2 主要符号与单位

符号	含义	单位
U_i	第 i 电场电压	kV
T_i	第 i 电场振打周期	s
C_{in}, C_{out}	入口、出口粉尘浓度	g/Nm ³ 、mg/Nm ³
Q	烟气流量	Nm ³ /h
L	粉尘质量负荷	kg/h
$P, \hat{P}$	实际、预测总功率	kW
K	无量纲去除指数	—
$B(T)$	振打引起的峰值增量代理	mg/Nm ³
ρ	四场振打相位叠加尺度	—
$q_{k,0.975}$	工况 k 历史 Mahalanobis 距离平方的经验 97.5% 分位数	—

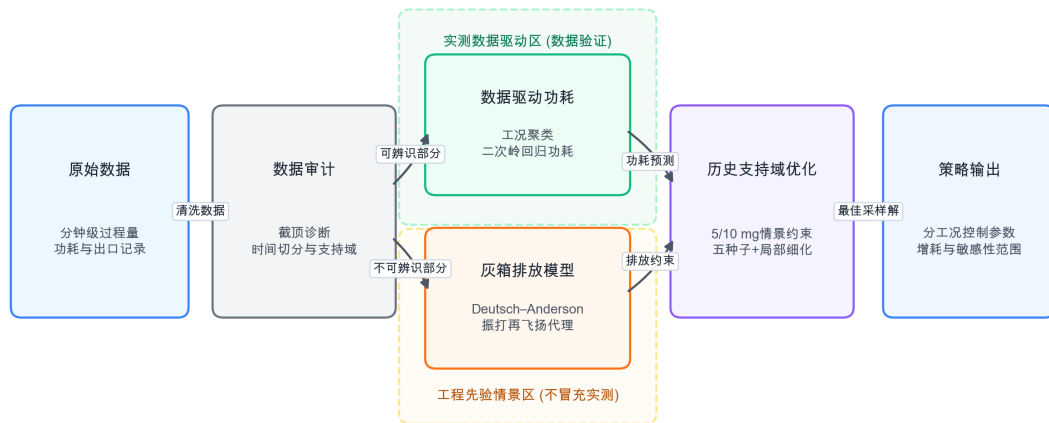


图 4 总体方法框架

4 问题一：影响关系与振打峰值

4.1 去除指数灰箱模型

Deutsch-Anderson 关系以指数形式描述电除尘效率，但其理想化形式忽略再飞扬、气流分布和粒径变化，因此适合作为灰箱骨架而不是无条件精确模型 [2-4]。White 的经典专著 [12] 与 Parker 的电除尘器电气运行专著 [13] 均强调电气运行、粉尘性质与振打之间的系统耦合，这也说明本文系数必须作为待现场标定的工程先验。定义

$$K = \ln \frac{1000C_{in}}{C_{out}}, \quad C_{out} = 1000C_{in}e^{-K} \quad (2)$$

对工况 k ，以历史中位电压 $U_{i,k}^{ref}$ 和缩放后的出口浓度中位数 $C_{0,k}$ 为锚点：

$$K_{0,k} = \ln \frac{1000C_{in,k}}{C_{0,k}} \quad (3)$$

考虑电压贡献和振打偏离损失，构造

$$K_k(U, T) = K_{0,k} + \eta \sum_{i=1}^4 \alpha_i \left[\left(\frac{U_i}{U_{i,k}^{ref}} \right)^2 - 1 \right] - \{G_k(T) - G_k(T^{ref})\} \quad (4)$$

$$G_k(T) = \sum_{i=1}^4 \beta_{i,k} \left(\frac{T_i}{T_{i,k}^*} + \frac{T_{i,k}^*}{T_i} - 2 \right), \quad \beta_{i,k} = \beta_i \left(\frac{L_k}{L_{med}} \right)^{1/2} \quad (5)$$

$G_k(T)$ 在 $T_i = T_{i,k}^*$ 处取最小值，表达周期过短与过长均可能不利：过短导致频繁再飞扬，过长导致积灰和单次脱落量上升。试验研究确实观察到振打再飞扬和最大允许周期之间的权衡 [6]，但本文系数并非由附件的振打事件估计。

本节灰箱排放模型涉及的全部系数、中心取值与证据状态如表 3 所示。

本节全部排放系数均为工程先验，不是由附件出口浓度估计得到的参数。

表 3 灰箱排放模型参数来源与证据状态

参数	中心取值	含义	来源	是否由附件识别
η	1.0	电压效应总体尺度	情景设定	否
α_i	四场权重	电场去除贡献	工程先验与消融	否
β_i	四场损失系数	周期偏离损失	工程情景	否
b_0	1.2	峰值尺度	工程情景	否
γ	1.35	周期—峰值指数	中心情景	否
ρ	1.0	相位叠加尺度	同相保守情景	否
T_i^*	由历史周期和负荷构造	参考振打周期	情景构造	否
s	0.08	安全裕度	工程情景	否

加入安全裕度 s 后，连续排放基值为

$$C_{base,k} = 1000C_{in,k} \exp[-K_k(U, T) + s] \quad (6)$$

4.2 振打峰值代理

附件只有振打周期设定，没有实际振打时刻、相位和秒级浓度，因此不能识别逐次峰值。为区分同相叠加上界与错相运行，本文引入相位尺度 ρ 和峰值指数 γ ：

$$B_k(T; \rho, \gamma) = b_0 \rho \left(\frac{L_k}{L_{med}} \right)^{0.8} \sum_{i=1}^4 w_i \left(\frac{T_i}{T_{i,k}^*} \right)^\gamma \quad (7)$$

构造保守峰值增量，情景峰值总浓度为

$$C_{peak,k} = C_{base,k} + B_k(T) \quad (8)$$

中心情景取 $\rho = 1$ 、 $\gamma = 1.35$ ，相当于四场峰值同相叠加的保守上界；错相情景取 $\rho = 0.65$ 或 0.80 。中心指数下，所有振打周期同步延长 10% 和 20% 时，单次峰值代理分别放大 13.74% 与 27.91%。灰箱参数组合敏感性同时扫描 $\gamma \in \{1.10, 1.35, 1.60\}$ ，避免把单一指数当作数据辨识结果。

振打周期与单次再飞扬峰值代理的关系如图 5 所示。周期延长会放大峰值代理，但该曲线仅表示工程情景，不是由附件中实际振打事件识别得到的因果关系。

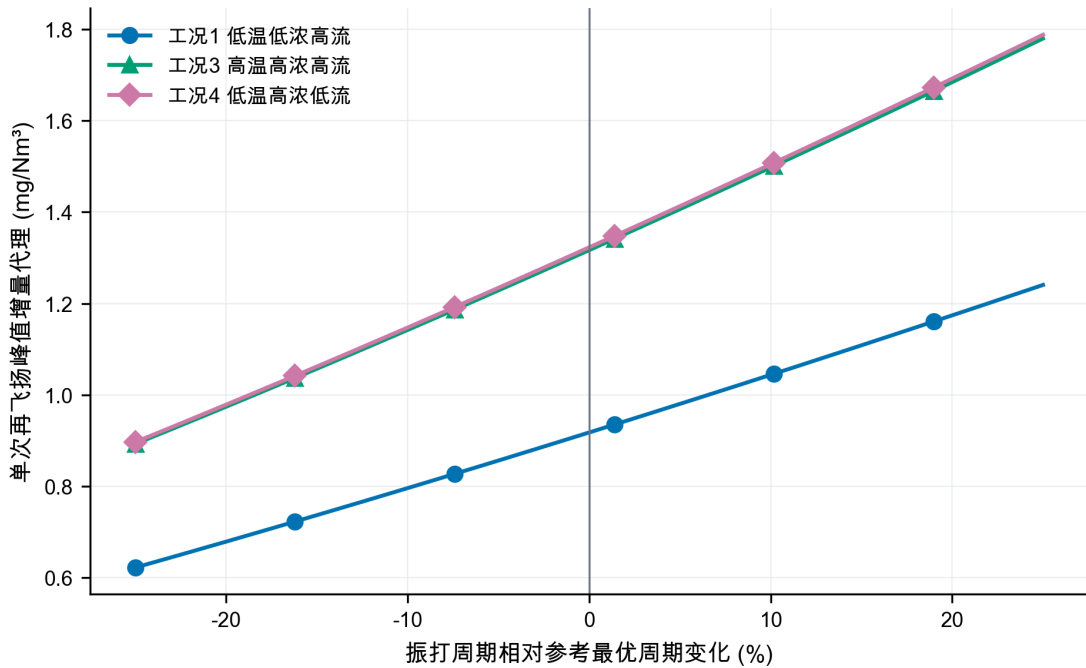


图 5 振打峰值情景机理

4.3 各因素方向性解释

在灰箱骨架中，固定其他条件时，提高 U_i 增加 K 并降低连续排放，但总功率随电压上升；固定 K 时，入口浓度提高使出口质量浓度同比提高；流量增加会缩短有效停留时间并提高质量负荷，通常要求更高捕集强度；温度会通过气体性质、粉尘比电阻和放电状态影响除尘性能 [3-4]。由于附件的出口浓度不可识别，温度和流量的排放弹性没有被强行拟合，而是通过工况分层和先验敏感性间接处理。由此，第一问的可靠答案分为两层：数据层只能报告“响应不可识别”；工程层可以给出上述方向和情景公式。

5 问题二：典型工况划分与协同优化

5.1 工况聚类

取训练期的入口温度、入口浓度和烟气流量三个外生变量，标准化后执行 K-means。K-means 通过最小化簇内平方和划分样本 [9]，轮廓系数同时衡量簇内紧密度与簇间分离度 [10]。比较 $k =$

2, 3, 4, 5, 6, 轮廓系数依次为 0.2963、0.3682、0.4084、0.3958、0.3934, 因此选择 $k = 4$ 。轮廓系数 0.4084 表明聚类分离度中等, 四类工况应理解为便于控制决策的统计工况分层, 而不是唯一的物理状态划分。

聚类数选择结果如图 6 所示。四类工况取得最高轮廓系数; 惯性随 k 增加持续下降, 因此不能仅依据惯性无限增加簇数。

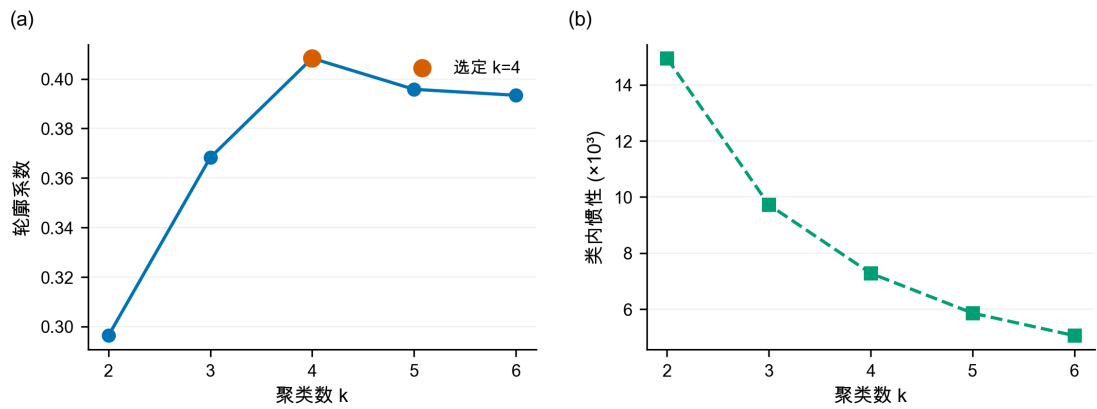


图 6 聚类数选择结果

训练期四类典型工况的分布如图 7 所示, 其中点的大小表示烟气流量, 从而同时呈现入口浓度、温度和流量三维信息。

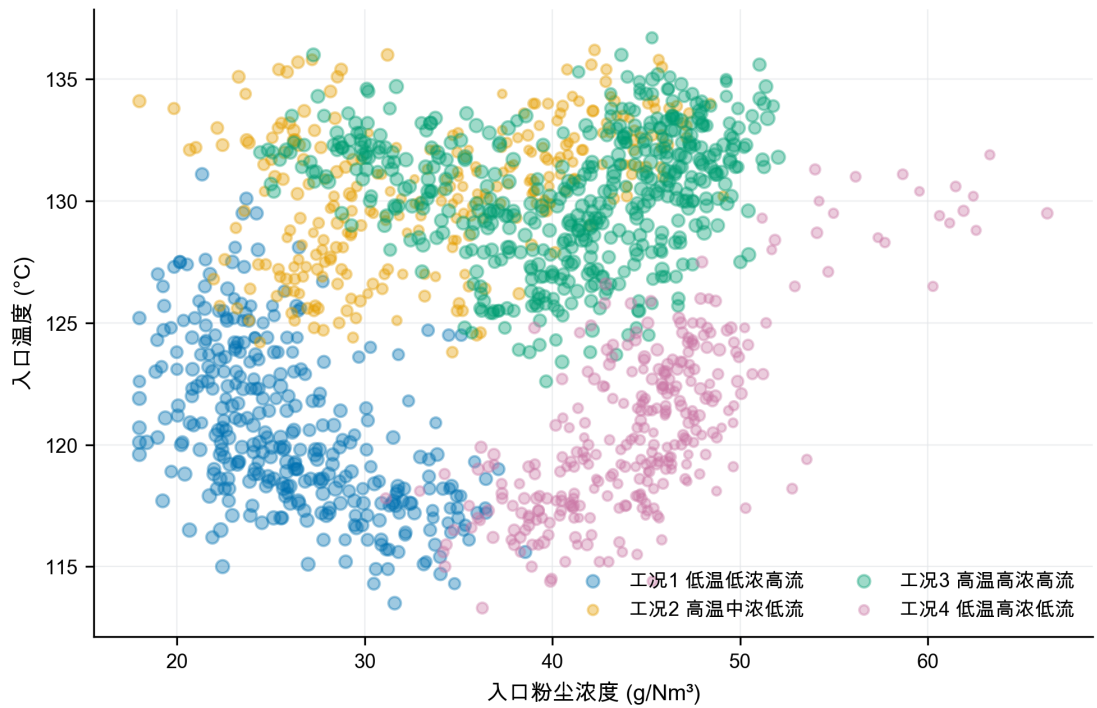


图 7 训练期典型工况分布

四类典型工况的统计画像如表 4 所示。

标化工况画像如图 8 所示。工况 3 与工况 4 的粉尘负荷相近, 但温度和流量不同, 适合用于检验分工况策略的必要性。

表 4 四类典型工况画像

工况	占比	温度/°C	入口浓度/(g/Nm³)	流量/(Nm³/h)	粉尘负荷/(kg/h)	历史总功率/kW
1 低温低浓度高流量	23.35%	120.65	26.08	478257	12460	1818.35
2 高温中浓度低流量	20.06%	130.32	33.72	443591	14922	1870.89
3 高温高浓度高流量	34.93%	130.41	40.71	480851	19560	1763.08
4 低温高浓度低流量	21.67%	120.69	44.75	439990	19673	1696.55

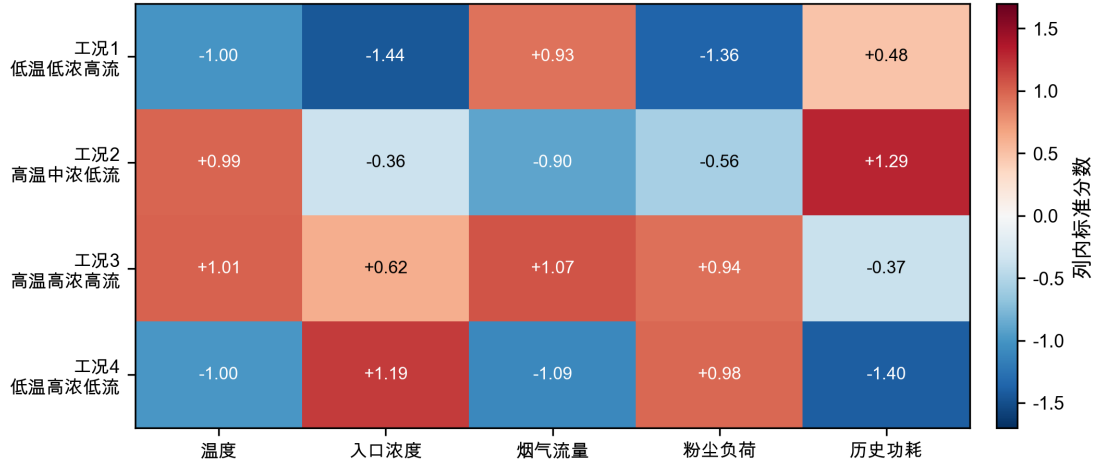


图 8 标准化工况画像

5.2 功率模型

电场总功率与电压存在非线性，且四个电场之间可能存在联动。构造特征向量

$$x = [U_1, \dots, U_4, U_1^2, U_1 U_2, \dots, U_4^2, T_1, \dots, T_4, H, C_{in}, Q] \quad (9)$$

标准化后采用岭回归 [11]

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \quad (10)$$

截距不惩罚。在线性岭与二次岭各自的候选正则强度中，按训练期顺序设置 3 个扩展窗口滚动验证，二次岭选得 $\lambda = 0.1$ ，线性岭选得 $\lambda = 10$ 。最终模型只用训练期拟合，验证期用于模型比较与误差保护，回顾性测试期不参与选择。验证集绝对误差的 90% 分位数为 11.78 kW，故每个候选同时报告点预测 $\hat{P}$ 与附加验证期误差后的功率 $\hat{P} + 11.78 \text{ kW}$ ；该附加量不改变候选排序，只用于表示功率预测误差的量级，不保证在后续日期仍具有 90% 的覆盖率。

均值预测、线性岭与二次岭在验证集和回顾性测试段的表现如表 5 所示。

表 5 功率模型基线与时间外推结果

模型	数据段	R^2	RMSE/kW	平均偏差/kW
均值预测	验证集	-5.3259	140.18	128.62
线性岭	验证集	0.9798	7.93	0.28
二次岭	验证集	0.9837	7.12	-1.87
均值预测	回顾性测试	-0.3733	83.51	-43.54
线性岭	回顾性测试	0.9180	20.41	-16.65
二次岭	回顾性测试	0.9571	14.76	-10.78

功率模型的回顾性测试结果如图 9 所示。二次岭回归保持较高预测一致性，并优于均值预测与

线性岭基线；左图虚线表示理想预测，右图 RMSE 采用对数纵轴以同时呈现三种模型的量级差异。

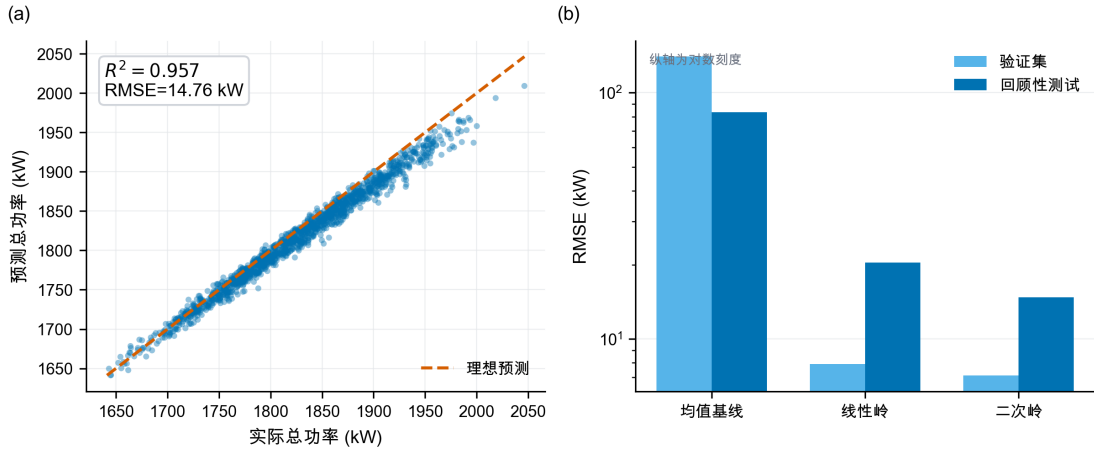


图9 功率模型预测结果

为识别功率预测主要依赖哪些变量，将二次岭模型的标准化系数按绝对值排序，如图 10 所示。电压、电压平方项及跨电场交互项提供了主要预测信息；由于各变量之间存在相关性，系数大小只用于解释模型的预测结构，不能直接解释为单变量因果效应。

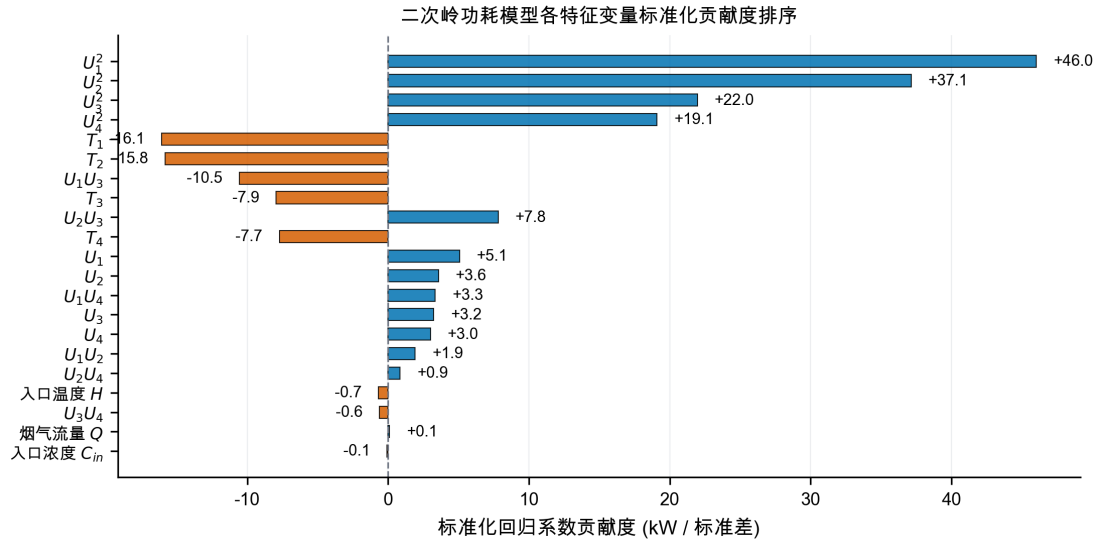


图10 二次岭功率模型的标准化系数

回顾性测试残差如图 11 所示。平均偏差为-10.78 kW，高功率区负偏差有所扩大；验证绝对误差 90% 分位数 11.78 kW 被用作验证期误差附加量。上线前仍需利用全新日期重新校准，不能把当前测试称为真正盲测。

各工况典型状态下的局部电压—功率响应如图 12 所示。曲线仅在历史电压附近绘制，表明在灰箱情景下，提高电压以满足更严格的排放约束会提高运行功率。

5.3 支持域与优化模型

设控制向量 $z = (U_1, \dots, U_4, T_1, \dots, T_4)$ 。历史 P05~P95 限制逐变量范围；同时用训练期工况 k 的均值 μ_k 和协方差 Σ_k 计算

$$d_M^2(z) = (z - \mu_k)^T \Sigma_k^{-1} (z - \mu_k) \leq q_{k,0.975} \quad (11)$$

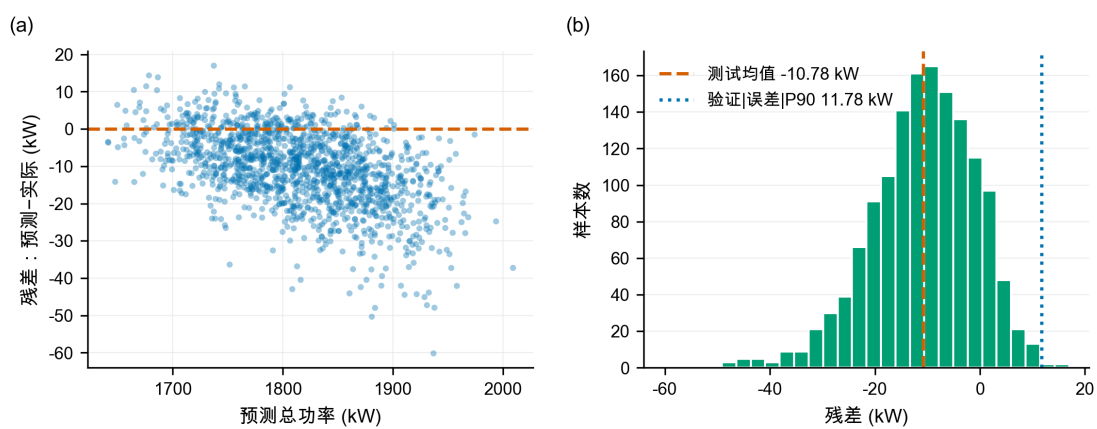


图 11 回顾性测试残差诊断

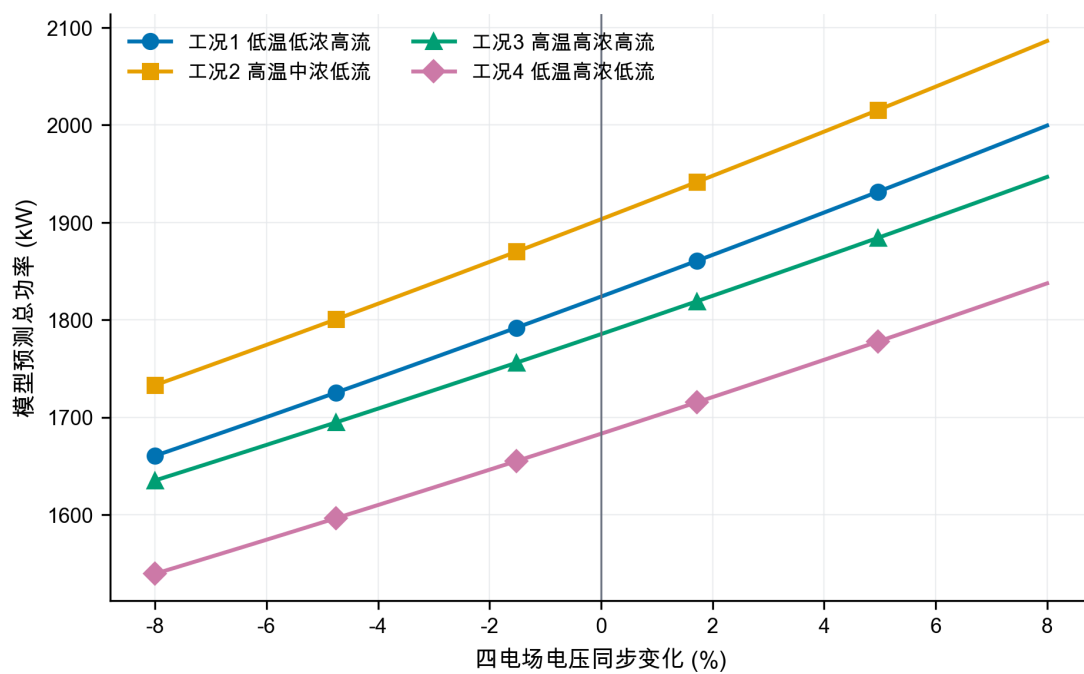


图 12 局部电压—功率响应

$q_{k,0.975}$ 直接取该工况历史距离平方的经验 97.5% 分位数, 四工况依次为 20.55、18.72、16.85 和 19.09, 不再借用多元正态假设下的统一卡方阈值。另加入

$$U_1 \geq U_3 + 3, \quad U_2 \geq U_4 + 3 \quad (12)$$

$$T_3 \geq T_1 + 100, \quad T_4 \geq T_2 + 100 \quad (13)$$

对每类工况和限值 C_{lim} 求解

$$\min_z \hat{P}_k(z) \quad (14)$$

$$\text{s.t. } C_{base,k}(z) + B_k(z) \leq C_{lim}, \quad z \in \Omega_k, \quad d_M^2(z) \leq q_{k,0.975} \quad (15)$$

程序对每个工况使用 5 个独立随机种子, 每个种子先生成 220000 组候选, 再围绕 10 与 5 mg/Nm³ 的初始最佳点各生成 30000 组局部高斯扰动。候选均经过经验支持域与工程排序过滤。文中参数称为“支持域内最佳采样可行解”; 五种子结果用于报告离散度, 局部细化用于检验随机库是否遗漏邻域改进。相关多电场节能研究也采用排放约束下的分场优化思想 [5,7], 但本文搜索边界和排放先验均按本题情景定义。

5.4 两档目标下的最佳采样可行方案

四类工况在 10 与 5 mg/Nm³ 两档排放目标下的中心情景最佳采样可行解如表 6 所示。

表 6 四工况两档目标的支持域内最佳采样可行解

工况	目标/(mg/Nm ³)	U_1 /kV	U_2 /kV	U_3 /kV	U_4 /kV	T_1 /s	T_2 /s	T_3 /s	T_4 /s	峰值排放/(mg/Nm ³)	预测总
1	10	55.4	53.5	51.8	48.0	267	268	465	465	9.997	
1	5	65.0	61.8	51.7	53.2	267	267	466	466	4.996	
2	10	57.8	53.1	52.9	48.9	258	258	448	450	9.995	
2	5	65.6	65.3	52.3	54.5	258	256	465	464	4.985	
3	10	52.6	55.0	49.2	44.1	253	253	456	466	9.996	
3	5	60.9	61.8	52.2	49.9	251	253	464	464	4.989	
4	10	51.8	50.8	48.7	46.6	267	268	473	464	9.989	
4	5	62.7	57.1	50.0	50.7	268	268	473	472	4.999	

其中 10 mg/Nm³ 四行直接回答问题二, 5 mg/Nm³ 四行为问题四与在线执行链提供完整参数。它们不是设备安全设定。将 10 mg/Nm³ 方案与同一功率模型在各工况“历史中位控制点”上的预测比较, 辅助节能幅度为 9.80%~12.43%; 之所以以“同一模型在历史中位控制点的预测”而非“历史实际功率均值”为基准, 是为了避免把模型偏差计入节能幅度, 但该比较仍受排放情景假设影响。表中为点预测功率, 预算时可另加 11.78 kW 验证期误差附加量。现场应用必须继续受二次电流、火花率、联锁和设定变化速率约束。

各推荐解与所在工况历史支持域边界的接近程度如表 7 所示。

数据表明: 工况 2 的 10 mg 解及工况 4 的 5 mg 解已高度贴近历史支持域的马氏距离上界($d_M^2/q_{0.975} > 90\%$)。这说明部分推荐控制点是在功率模型与支持域边界的共同作用下推至边界附近, 实际工业应用时仍需通过现场阶跃试验重新标定, **不可直接将支持域边界解当作绝对安全解。**

推荐解与经验支持域边界的距离对比如图 13 所示。左图给出边界相对距离, 右图并列展示推荐解马氏距离平方与各工况经验阈值; 两种视图共同显示所有解均未越界, 但工况 2 的 10 mg 解和工况 4 的 5 mg 解已进入 90% 贴近区。

表 7 推荐解的历史支持域边界接近度审计

工况与排放限值	马氏距离平方 d_M^2	经验 97.5% 分位数 $q_{0.975}$	边界相对距离 $d_M^2/q_{0.975}$	边界状态评估
工况 1, 10 mg	10.89	20.55	53.0%	内部良好
工况 1, 5 mg	10.11	20.55	49.2%	内部良好
工况 2, 10 mg	17.52	18.72	93.6%	贴近边界
工况 2, 5 mg	10.96	18.72	58.5%	内部良好
工况 3, 10 mg	8.19	16.85	48.6%	内部良好
工况 3, 5 mg	14.50	16.85	86.1%	接近边界
工况 4, 10 mg	7.40	19.09	38.8%	内部良好
工况 4, 5 mg	18.16	19.09	95.1%	贴近边界

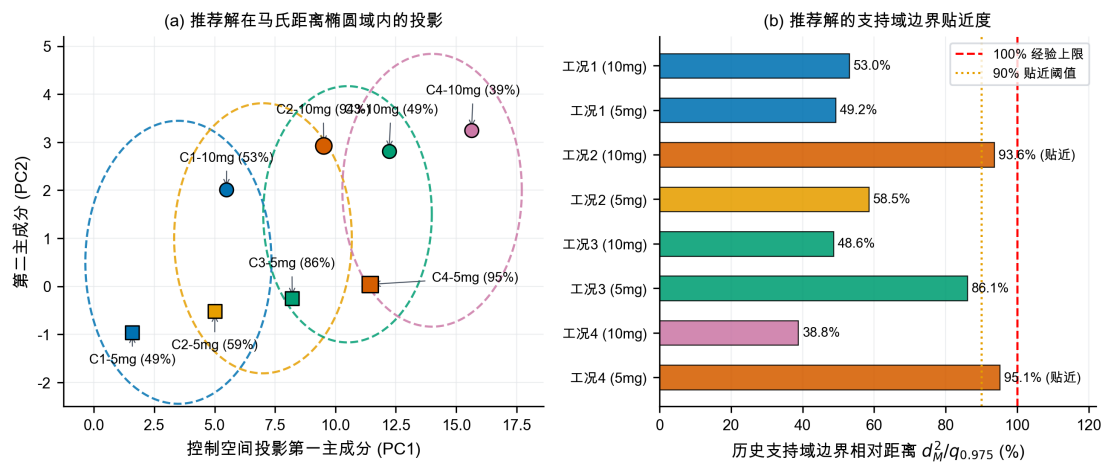


图 13 推荐解的历史支持域边界接近度

四类工况在 10 和 5 mg/Nm³ 限值下的中心情景最佳采样预测功率如图 14 所示。更严格的排放限值在全部工况下均提高预测功率。

6 问题三：代表工况的控制策略比较

选取工况 1 和工况 3。二者流量都较高，但入口浓度和粉尘负荷差异明显：工况 1 平均负荷 12460 kg/h，工况 3 为 19560 kg/h，可用于比较低负荷与高负荷策略。

两类代表工况在两档排放限值下的中心情景控制参数如表 8 所示。

表 8 代表工况在两档限值下的中心情景控制参数

工况与限值	U_1/kV	U_2/kV	U_3/kV	U_4/kV	T_1/s	T_2/s	T_3/s	T_4/s	预测总功率/kW
工况 1, 10 mg	55.4	53.5	51.8	48.0	267	268	465	465	1629.63
工况 1, 5 mg	65.0	61.8	51.7	53.2	267	267	466	466	1851.86
工况 3, 10 mg	52.6	55.0	49.2	44.1	253	253	456	466	1595.37
工况 3, 5 mg	60.9	61.8	52.2	49.9	251	253	464	464	1813.76

工况 1 从 10 收紧到 5 mg/Nm³ 时， U_1, U_2 分别提高 9.6 和 8.3 kV， U_3 近似不变， U_4 提高 5.2 kV；工况 3 的四场增量依次为 8.3、6.8、3.0 和 5.8 kV。因而“前级优先”不能表述成“后级无需调整”。进一步把去除贡献权重改为等权、弱前级和中心前级三种形状后，前两电场仍承担 70.8%~73.7% 的正向加权电压增量，说明该规律不完全由中心权重预设，但其强度与具体后级补偿仍属于情景发现。

为把上述比较转化为可执行判据，本文按排放构成与支持域状态给出控制优先级决策规则，如表 9 所示。

上述决策规则均基于灰箱情景推导，现场实施时应以在线监测数据为准。

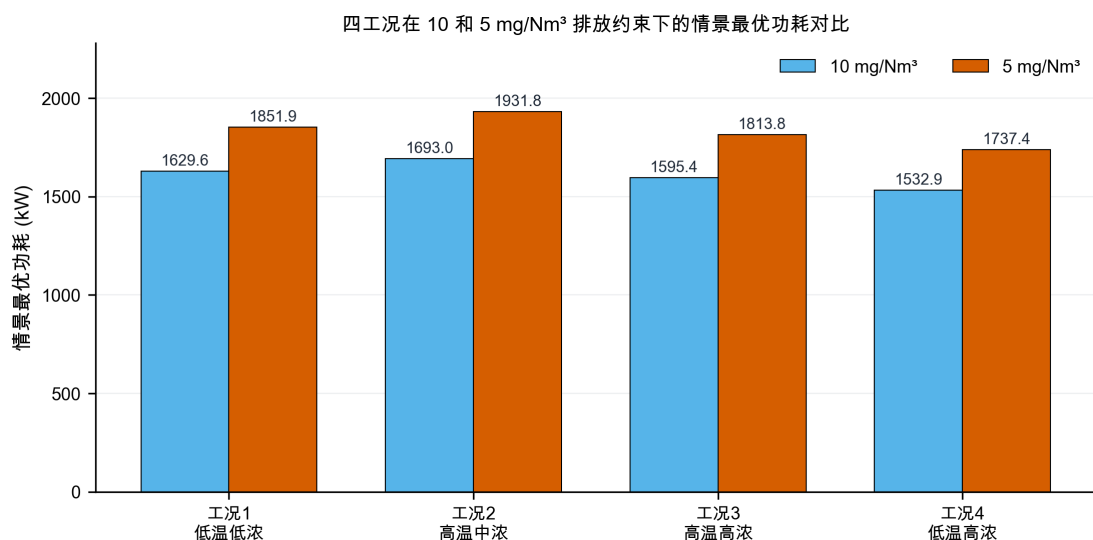


图 14 两档排放限值下的最佳采样预测功率

表 9 控制优先级决策规则

状态特征	优先动作	原因
连续排放基值接近限值，振打峰值占比较低 峰值增量占比较高，周期明显长于参考周期 周期过短、振打频繁	优先小步提高前级电压 优先缩短过长周期并实施错相振打 适度延长周期	主要矛盾是连续捕 主要矛盾是单次积 减少再飞扬次数
高粉尘负荷且基值和峰值均高 前级电压已接近支持域上限 超出历史支持域或出现火花、联锁风险	先错相、再调整周期，最后小步提升前级电压 使用后级电场补偿 停止优化，回退现场安全设定	先控制峰值，再补 避免前级单独承担 数学可行不等于设

两档排放限值下的四电场中心情景电压如图 15 所示。严格约束主要抬升前两电场，同时 U_4 在部分工况承担明显补偿，因此不能将策略绝对化为“只调前级”。

为突出代表工况内部四电场的协同变化，将工况 1 与工况 3 的电压和振打周期分别绘制为控制剖面，如图 16 所示。限值由 10 收紧至 5 mg/Nm³ 时，两类工况均以前级电压抬升为主，但 U_4 仍有补偿；振打周期的变化则更依赖工况，说明电压调节与振打调节不能拆成两个独立单变量策略。

两档排放限值下的中心情景最佳采样振打周期如图 17 所示。周期变化不像电压那样单向，而是在积灰损失与再飞扬峰值之间重新平衡；前级短、后级长的顺序保持不变。

在线实施采用 **TARGET-10**、**TARGET-5** 与 **FALLBACK** 三态。排放目标由上级指令或管理规定选择，不设自动切换阈值——因为本文灰箱模型未经现场标定，不具备在线自动判定排放状态的精度。**TARGET-10** 与 **TARGET-5** 分别读取表 6 中对应工况、对应排放目标的控制参数；表 8 仅用于问题三的代表工况比较。控制量按斜坡接近目标，试运行情景暂取电压不超过 0.5 kV/min、振打周期不超过 5 s/min，但现场 DCS、联锁或设备制造商给出的更严限制具有优先权。四场振打先按 $\rho = 1$ 同相上界核验，再以 $\rho = 0.80$ 和 0.65 评估错相收益。**FALLBACK** 仅在传感器失效、火花率越限、工况超出支持域或联锁触发时启用，此时回到现场确认的安全设定；未经现场标定的模型偏差不直接作为自动回退阈值。

7 问题四：10 收紧至 5 mg/Nm³ 的功率代价

7.1 分工况结果

各工况从 10 收紧至 5 mg/Nm³ 的中心情景功率增幅如表 10 所示。
按训练期工况占比 π_k 加权，

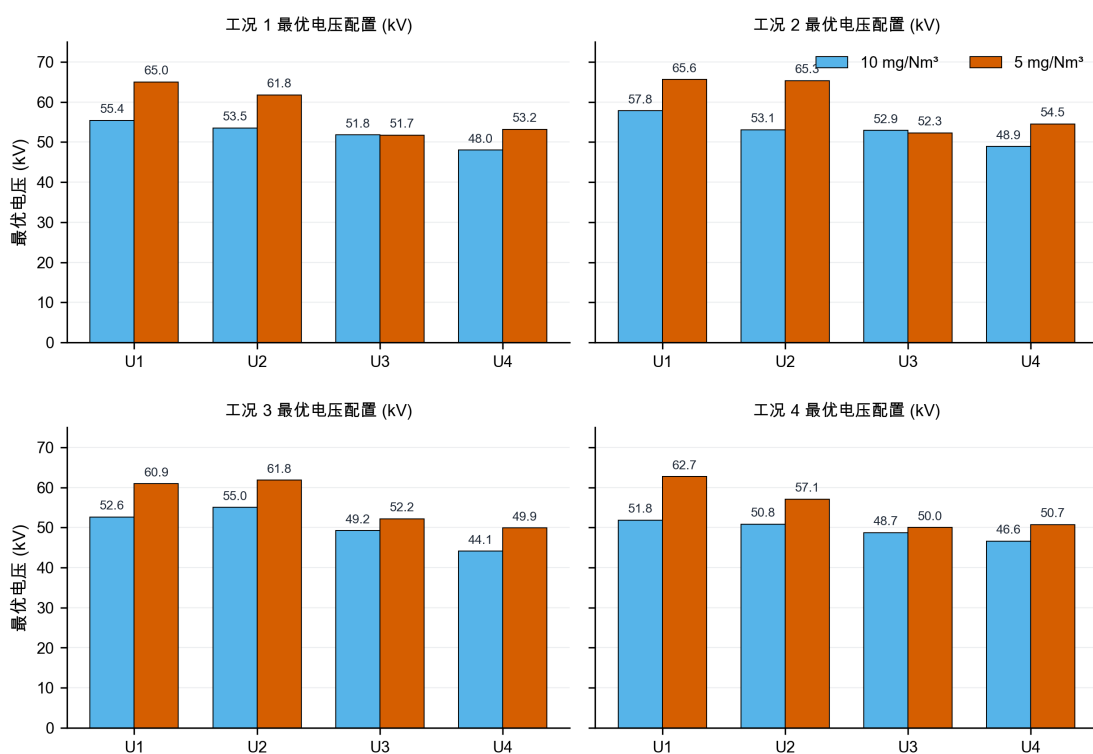


图 15 两档排放限值下的电压配置

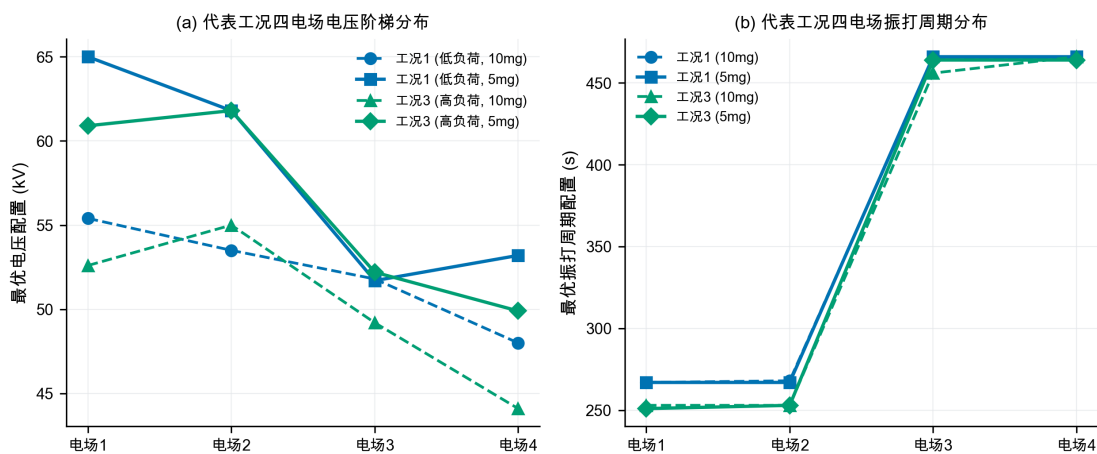


图 16 代表工况四电场控制参数协同剖面

表 10 中心情景下 10 收紧至 5 mg/Nm³ 的分工况功率增幅

工况	10 mg 预测功率/kW	5 mg 预测功率/kW	功率增幅
1 低温低浓度高流量	1629.63	1851.86	13.64%
2 高温中浓度低流量	1693.00	1931.84	14.11%
3 高温高浓度高流量	1595.37	1813.76	13.69%
4 低温高浓度低流量	1532.90	1737.39	13.34%

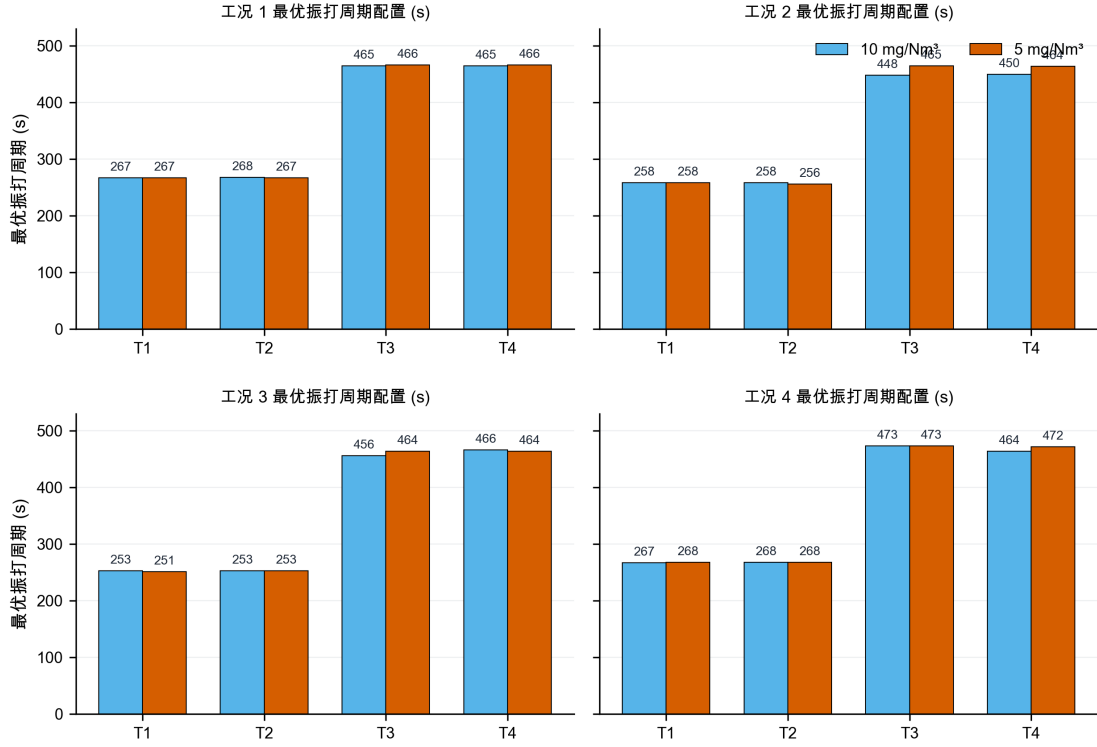


图 17 两档排放限值下的振打周期

$$\bar{P}_{10} = \frac{\sum_k \pi_k P_{k,10}}{\sum_k \pi_k} = 1609.42 \text{ kW} \quad (16)$$

$$\bar{P}_5 = 1829.79 \text{ kW}, \quad \Delta P = \frac{\bar{P}_5 - \bar{P}_{10}}{\bar{P}_{10}} \approx 13.7\% \quad (17)$$

加上 11.78 kW 验证期误差附加量后，两档加权功率分别为 1621.19 和 1841.57 kW；由于两者使用同一附加量，该比例不作为严格上界，而只用于功率预算。

四类工况从 10 收紧至 5 mg/Nm³ 的中心情景功率增幅如图 18 所示。分工况功率增幅为 13.34%~14.11%，工况加权点估计约 13.7%；在相同运行时长下，该比例也等于电耗增幅。该点值必须与灰箱参数组合情景范围同时报告。

7.2 峰值约束的作用

中心情景最佳采样解中，10 mg/Nm³ 方案的连续基值为 8.45~9.04 mg/Nm³，振打峰值增量为 0.95~1.54 mg/Nm³；5 mg/Nm³ 方案的连续基值降至 3.45~4.04 mg/Nm³，为振打峰值留出约 0.95~1.55 mg/Nm³ 裕度。因此不能只令平均或基值低于限值，否则振打时刻仍可能超限。

中心情景最佳采样解的连续排放基值与振打峰值增量如图 19 所示。图中虚线表示情景限值，所有柱形均为模型推演结果，而非实测排放数据。

7.3 高浓度工况建议

工况 3 和 4 的粉尘负荷最高。对于这两类工况，建议：

1. 以 U_1, U_2 承担主体捕集，避免四个电场同步大幅抬升造成不必要的功率增加；
2. 保留 U_3, U_4 的精除余量，出口趋势接近阈值时再小步提高；

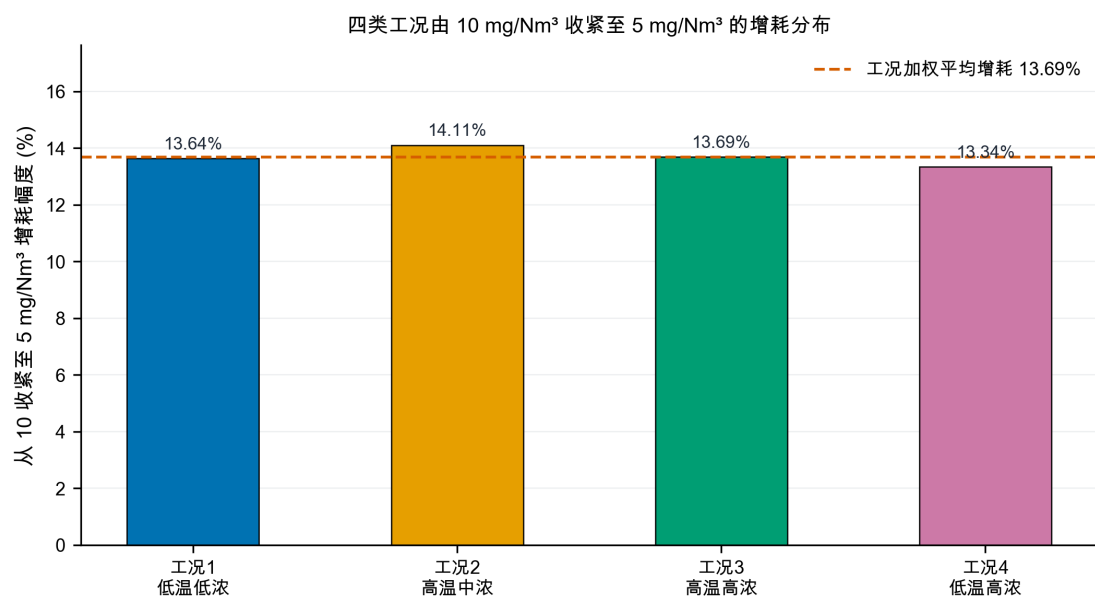


图 18 分工况功率增幅比较

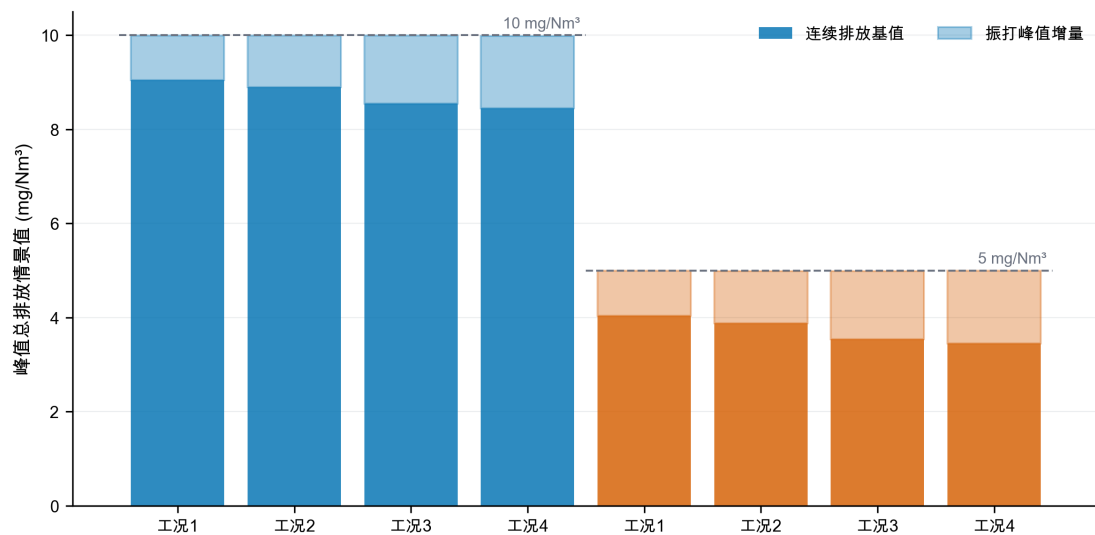


图 19 排放约束分解

3. 四电场振打错相，避免峰值叠加；高负荷下不宜为降低振打次数而持续延长周期；
4. 将秒级未封顶出口浓度、实际振打事件、二次电流和火花率接入在线控制；
5. 设置人工审核、变化速率限制和自动回退，先影子运行再闭环投用。

8 模型检验、敏感性与稳健性

8.1 多种子稳定性与局部细化

对 5 个独立随机种子分别执行“每工况 220000 组全域采样+每个初始最佳点 30000 组局部扰动”。八个工况—限值组合中，最佳采样预测功率的最大变异系数为 0.181%，最大极差占均值 0.464%；局部细化相对初始随机库的改善为 0.18%~0.99%。这说明结果对种子不敏感，且邻域细化仍有小幅价值，但不能证明数学意义上的全局最优，因此全文统一使用“最佳采样可行解”。

五种子离散性与局部细化收益如图 20 所示。左图以各组合五种子中位数为基准，右图给出局部细化相对初始随机库的改善；数值稳定不等价于获得全局最优解。

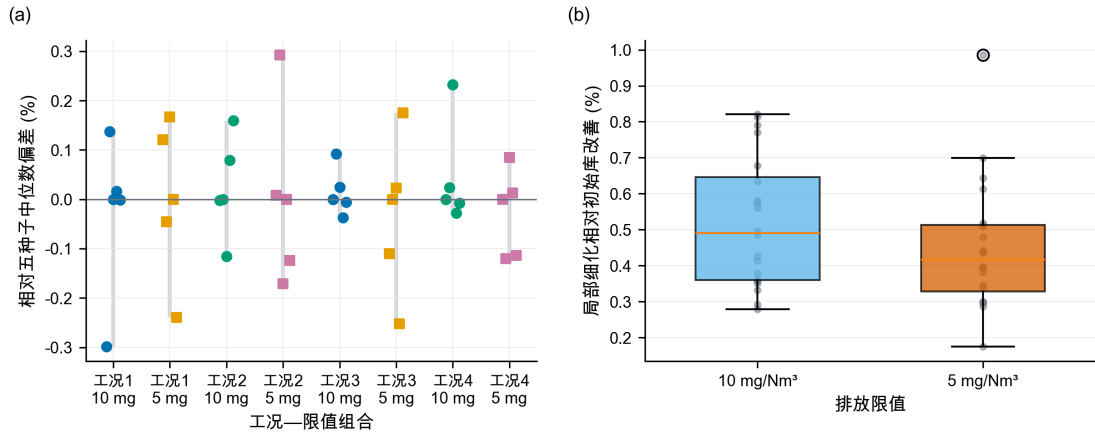


图 20 随机搜索稳定性与局部细化收益

8.2 灰箱参数组合敏感性

总体尺度扫描仍保留 27 组：电压效应尺度 $\eta \in \{0.8, 1.0, 1.2\}$ 、振打峰值尺度 $b_0 \in \{0.9, 1.2, 1.5\}$ 、安全裕度 $s \in \{0, 0.08, 0.15\}$ 。其中 26 组全工况可行，功率增幅为 10.40%~18.45%。更重要的是，灰箱参数组合扫描涵盖出口缩放 0.08~0.12、三种场权重形状、三个峰值指数、三个相位尺度和三个参考周期负荷指数，共 405 组且全部可行；灰箱参数组合情景功率增幅为 11.37%~14.31%，中位数 12.99%。两类范围都是先验网格上的灰箱参数组合情景范围，不是统计置信区间。

405 组灰箱参数组合情景的功率增幅分布如图 21 所示。出口缩放与振打相位会系统改变功率增幅，中心点约 13.7% 应与 11.37%~14.31% 的灰箱参数组合情景范围结合解释。

电场贡献权重消融结果如图 22 所示。在等权、弱前级和中心前级三种设定下，前两电场仍承担 70.8%~73.7% 的正向电压增量；该结果支持有条件的“前级优先”，同时说明 U_4 补偿不可忽略。

8.3 稳健性结论

稳健性证据分四层：轮廓系数支持四工况分层；基线与滚动验证支持功率模型选择；五种子与局部细化支持数值搜索稳定；灰箱参数组合情景与场权重消融检验灰箱结论对建模选择的依赖。中心功率增幅约 13.7%，但灰箱参数组合情景中位数为 12.99%，二者差异正说明不能只报告单点。验

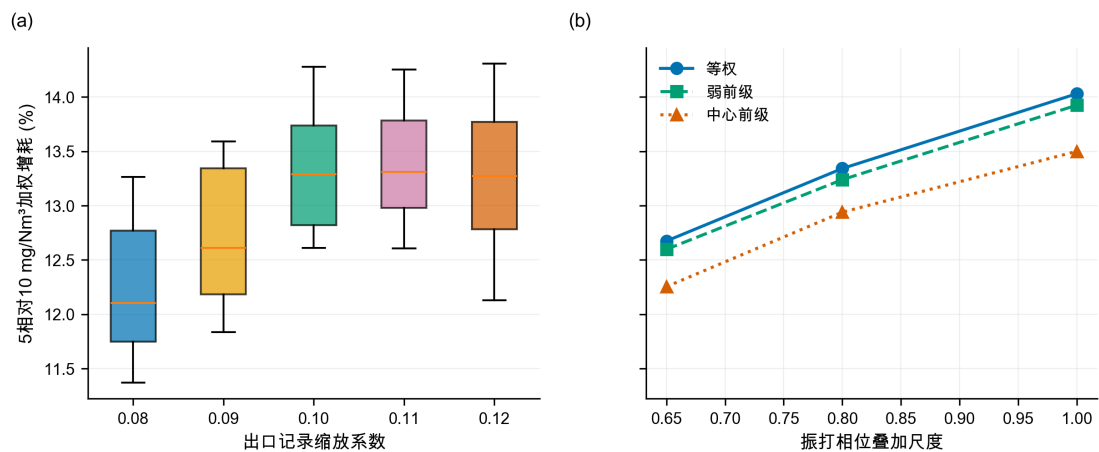


图 21 功率增幅敏感性分布

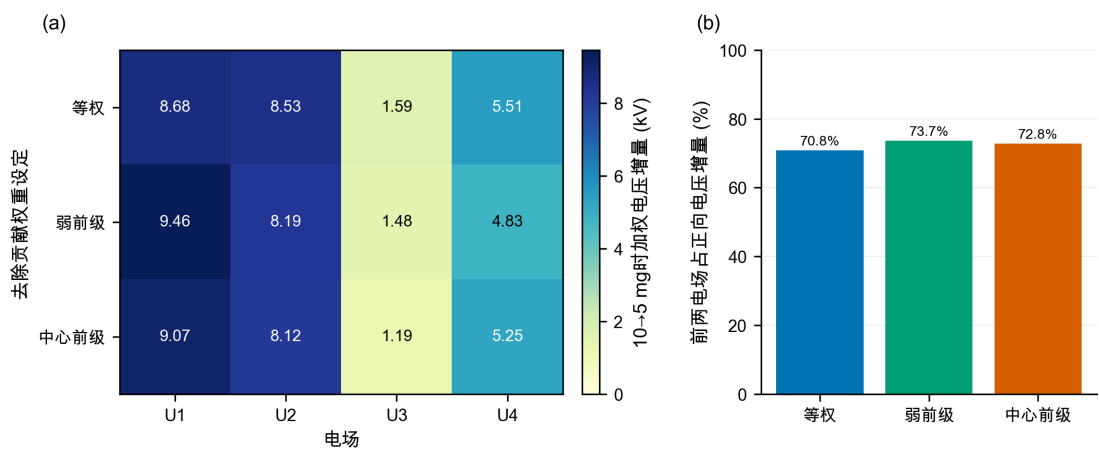


图 22 电场贡献权重消融结果

证期误差附加量、测试偏差和情景范围也不能把灰箱模型变成实证模型，现场补采与盲测仍是闭环前置条件。

9 模型评价、实施方案与局限

9.1 模型优点

第一，设置可识别性门禁，避免用复杂算法拟合封顶噪声。第二，工况聚类不使用出口浓度和总功率，减少以结果反向定义工况的偏差。第三，功率模型采用均值、线性、二次基线和滚动验证，并报告偏差与验证期误差附加量。第四，支持域使用逐变量分位数和经验 Mahalanobis 阈值，降低不合理外推。第五，优化使用五种子与局部细化，且不冒称全局最优。第六，405 组灰箱参数组合情景、场权重消融和振打相位尺度把关键先验显式化。

9.2 模型局限

最主要的局限是出口浓度缩放、灰箱系数和振打峰值结构均未由附件识别；11.37%~14.31% 只是给定网格上的灰箱参数组合情景范围，不是统计置信区间，也不覆盖所有物理模型。历史 P05~P95 和经验 Mahalanobis 阈值不是设备安全边界；模型没有二次电流、分场功率、火花率、极板积灰状态和连锁信息；分钟级数据不足以观察逐次振打峰值。回顾性测试数据曾被前期处理流程使用，不能视为完全未触碰的盲测。因而，本文控制参数只能作为试验起点，**不得直接下发生产设备**。

9.3 上线与补采方案

建议按四阶段部署。第一阶段只做影子计算，验证工况识别和 TARGET-10/TARGET-5 参数表的正确加载；第二阶段在每类工况内执行单因素小步阶跃，并由现场确定真实电压和周期斜坡上限；第三阶段用 1~5 s 未封顶出口浓度、实际振打事件、二次电流、火花率和分场功率重新标定灰箱参数及相位尺度；第四阶段冻结模型，在全新日期盲测，通过后才允许闭环。任何超支持域、传感器失效或连锁事件都触发 FALLBACK。EPA 技术资料和相关实验均强调气流、收尘面积及再飞扬对性能评价的重要性 [2-3,6]，故补采应优先补齐这些状态链。

10 结论

本文对四个问题得到如下结论。

- 问题一：可识别性与方向性关系。**附件出口浓度在 50 mg/Nm³ 附近显著截顶，无法从数据实证识别控制量对真实排放及逐次振打峰值的影响。灰箱模型只提供条件式方向与试验假设；同相振打是情景上界，错相收益需由事件级数据标定。
- 问题二：工况划分、功率模型与 10 mg 方案。**基于入口温度、入口浓度和流量识别出 4 类典型工况，轮廓系数 0.4084。滚动验证选择的二次岭优于均值与线性基线，验证 $R^2 = 0.9837$ 、RMSE=7.12 kW；验证期误差附加量为 11.78 kW。中心情景下，10 mg/Nm³ 的四工况最佳采样预测功率为 1532.90~1693.00 kW；五种子最大变异系数 0.181%，局部细化最多改善 0.986%，支持数值稳定但不构成全局最优证明。
- 问题三：控制优先级。**等权、弱前级和中心前级消融下，前两电场承担 70.8%~73.7% 的正向电压增量，故可保留有条件的“前级优先”；后级尤其 U_4 仍承担情景相关补偿，振打周期需与电压协同调整。
- 问题四：5 mg 方案与功率代价。**排放目标由 10 收紧至 5 mg/Nm³ 时，中心情景加权功率由 1609.4 增至 1829.8 kW，功率增幅约 13.7%；405 组灰箱参数组合情景范围为 11.37%~14.31%，中位

数 12.99%。在相同运行时长下，该功率增幅也等于电耗增幅。上述数值均为情景推演，必须在补采未封顶秒级排放、振打事件和电气状态后重新标定。

参考文献

- [1] 中华人民共和国生态环境部. 水泥工业大气污染物排放标准: GB 4915—2013 (含 2025 年修改单)[EB/OL]. (2013-12-27)[2026-08-06]. <https://www.mee.gov.cn/ywgz/fgbz/bz/bzwb/dqhjbh/dqgdwrywrwpfbz/201312/t20131227/20131227145826001.htm>
- [2] OGLESBY S Jr, NICHOLS G B. *A Manual of Electrostatic Precipitator Technology: Part I—Fundamentals*[R/OL]. Birmingham: Southern Research Institute, 1970[2026-08-06]. <https://nepis.epa.gov/Exe/ZyPURL.cgi?DockKey=9400034C.TXT>
- [3] SZABO M F, SHAH Y M, SCHLIESSER S P. *Inspection Manual for Evaluation of Electrostatic Precipitator Performance*[R/OL]. Washington, DC: U.S. Environmental Protection Agency, 1981[2026-08-06]. <https://nepis.epa.gov/Exe/ZyPURL.cgi?DockKey=9400034C.TXT>
- [4] MIZUNO A. Electrostatic precipitation[J]. IEEE Transactions on Dielectrics and Electrical Insulation, 2000, 7(5): 615-624. DOI: 10.1109/94.879357.
- [5] LI Z, SHAO C, AN Y, et al. Energy-saving optimal control for a factual electrostatic precipitator with multiple electric-field stages based on GA[J]. Journal of Process Control, 2013, 23(8): 1041-1051. DOI: 10.1016/j.jprocont.2013.06.007.
- [6] NAVARRETE B, VILCHES L F, RODRIGUEZ-GALAN M, et al. A pilot scale study of the rapping reentrainment and fouling in electrostatic precipitation[J]. Environmental Progress & Sustainable Energy, 2015, 34(1): 7-14. DOI: 10.1002/ep.11934.
- [7] ZHENG C, ZHAO Z, GUO Y, et al. A real-time optimization method for economic and effective operation of electrostatic precipitators[J]. Journal of the Air & Waste Management Association, 2020, 70(7): 708-720. DOI: 10.1080/10962247.2020.1767227.
- [8] FU Y, SU Z. Deep neural network based concentration prediction model of dry electrostatic precipitator with a modified PID optimizer[J]. Journal of Physics: Conference Series, 2022, 2189: 012018. DOI: 10.1088/1742-6596/2189/1/012018.
- [9] MACQUEEN J. Some methods for classification and analysis of multivariate observations[C]//Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability. Berkeley: University of California Press, 1967, 1: 281-297.
- [10] ROUSSEEUW P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J]. Journal of Computational and Applied Mathematics, 1987, 20: 53-65. DOI: 10.1016/0377-0427(87)90125-7.
- [11] HOERL A E, KENNARD R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55-67. DOI: 10.1080/00401706.1970.10488634.
- [12] WHITE H J. *Industrial Electrostatic Precipitation*[M]. Reading, MA: Addison-Wesley, 1963.
- [13] PARKER K R. *Electrical Operation of Electrostatic Precipitators*[M]. London: Institution of Electrical Engineers, 2003.

附录 A 程序求解流程与源程序结构

A.1 求解流程

输入：分钟级数据、5个随机种子、排放限值 C_{lim}

1. 按时间划分训练/验证/回顾性测试；
2. 在训练期对Temp、 C_{in} 、Q标准化，K-means取 $k=4$ ；
3. 比较均值、线性岭、二次岭，按3个滚动窗口选择正则强度；

4. 对每个工况与随机种子：

4.1 计算P05、P95与经验Mahalanobis 97.5%阈值；

4.2 生成220000组全域候选；

4.3 在10/5 mg初始最佳点附近各生成30000组局部候选；

4.4 过滤支持域与工程排序，选择预测总功率最低的情景可行候选；

5. 扫描27组总体参数情景和405组灰箱参数组合情景；

6. 执行场权重消融、多种子离散和动态控制表输出；

输出：控制参数、功率增幅、验证期误差附加量、消融、敏感性、图表与运行摘要。

A.2 源程序清单与模块结构

完整可运行源程序随电子支撑材料提交，各程序的职责与顶层定义如表 A1 所示，完整代码见附录 D。

表 A1 源程序清单与顶层模块结构

源文件	职责	主要顶层定义
src/scenario_model.py	功率模型、灰箱排放情景、支持域优化与敏感性	voltage_polynomial、RidgePow
src/paper_figures.py	22 幅正文图与图注追踪表生成	fig01_outlet_diagnosis—fig18
src/plot_utils.py	统一绘图样式与 PNG/PDF 双格式导出	set_paper_style、finish_axes
src/build_paper_pdf.py	由终稿 Markdown 生成符合竞赛格式规范的 XeLaTeX 提交稿	convert、convert_table、buil

正文 Markdown 中的标记为修订补丁的定位锚点,由 build_final_deliverables.py 的 strip_work_comments 在装配提交稿时统一移除，不出现在最终 DOCX/PDF 中。

支撑数据、结果 CSV/JSON、图注追踪表、逐点回复与 AI 工具使用详情见附录 C 清单。

附录 B 结果适用性声明

本文功率模型、工况划分及数据质量统计直接来源于附件；出口浓度缩放、排放灰箱系数、振打峰值系数以及由此得到的 5/10 mg/Nm³ 最佳采样可行参数和功率增幅均属于情景推演。若数据提供方确认原记录不存在小数点问题，或提供新的未封顶排放数据，则应撤销当前排放情景并重新标定，不得沿用本文情景数值。

附录 C 支撑材料与 AI 工具使用声明

支撑材料清单：原始处理数据包；results 目录下功率基线、滚动验证、中心控制、多种子稳定性、结构情景、场权重消融、动态控制表和运行摘要；figures_paper 目录下 21 幅 PNG 预览、21 幅 PDF 矢量图及 figure_manifest.csv；src 目录下 4 个源程序；integrity 目录下确定性核验结果；revision_round1 目录下修订补丁、应用报告、修改追踪表与逐点回复。

AI 工具使用声明（提交前由参赛队核验确认）：本稿坚持参赛队人工主导。题意分析、模型构建、公式推导、参数设定、程序审阅、结果复算和论文定稿由参赛队承担；OpenAI Codex（使用日期 2026-08-05 至 2026-08-06）仅用于少量代码排错、图表版式检查、文字校对与格式核验。AI 输出均经人工筛选和复核，未替代现场数据，也未把不可识别的排放关系改写为实证结论。详细用途和人工核验记录另见过程研讨目录中的 AI 工具使用详情.md。

附录 D 完整源程序

本附录给出生成本文全部结果与图表的完整可运行源程序，与电子支撑材料 src/ 目录一致。

D.1 scenario_model.py

```
1  #!/usr/bin/env python3
2  """Reproducible scenario model for the cement ESP problem.
3
4  The power model and condition profiles are learned from the supplied data.
5  The emission/rapping layer is an explicitly labelled engineering scenario,
6  because the recorded outlet concentration is not identifiable.
7  """
8
9  from __future__ import annotations
10
11  import json
12  import math
13  from pathlib import Path
14
15  import matplotlib.pyplot as plt
16  import numpy as np
17  import pandas as pd
18
19
20  SEED = 20260805
21  SEEDS = [20260805, 20260806, 20260807, 20260808, 20260809]
22  ROOT = Path(__file__).resolve().parents[1]
23  DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
24  CLUSTER_EVAL_FILE = ROOT / "data" / "condition_cluster_evaluation.csv"
25  OUT_DIR = ROOT / "results"
26  FIG_DIR = ROOT / "figures"
27
28  U_COLS = [f"U{i}_kV" for i in range(1, 5)]
29  T_COLS = [f"T{i}_s" for i in range(1, 5)]
30  COND_COLS = ["Temp_C", "C_in_gNm3", "Q_Nm3h"]
31  POWER_FIELDS = U_COLS + T_COLS + COND_COLS
32
33
34  def _r2(y: np.ndarray, pred: np.ndarray) -> float:
35      return float(1.0 - np.sum((y - pred) ** 2) / np.sum((y - y.mean()) ** 2))
36
37
38  def _rmse(y: np.ndarray, pred: np.ndarray) -> float:
39      return float(np.sqrt(np.mean((y - pred) ** 2)))
40
41
42  def voltage_polynomial(u: np.ndarray) -> np.ndarray:
43      """Equivalent to PolynomialFeatures(degree=2, include_bias=False) for 4 voltages."""
44      cols = [u[:, i] for i in range(4)]
45      for i in range(4):
46          for j in range(i, 4):
47              cols.append(u[:, i] * u[:, j])
48      return np.column_stack(cols)
49
50
51  class RidgePowerModel:
52      def __init__(self, alpha: float = 1e-3, quadratic: bool = True):
53          self.alpha = alpha
54          self.quadratic = quadratic
55          self.mean: np.ndarray | None = None
```

```

56     self.std: np.ndarray | None = None
57     self.beta: np.ndarray | None = None
58
59     def design(self, frame: pd.DataFrame) -> np.ndarray:
60         u = frame[U_COLS].to_numpy(float)
61         remainder = frame[T_COLS + COND_COLS].to_numpy(float)
62         voltage = voltage_polynomial(u) if self.quadratic else u
63         return np.column_stack([voltage, remainder])
64
65     def fit(self, frame: pd.DataFrame, y: np.ndarray) -> None:
66         x = self.design(frame)
67         self.mean = x.mean(axis=0)
68         self.std = x.std(axis=0)
69         self.std[self.std == 0] = 1.0
70         z = (x - self.mean) / self.std
71         zz = np.column_stack([np.ones(len(z)), z])
72         penalty = np.eye(zz.shape[1]) * math.sqrt(self.alpha)
73         penalty[0, 0] = 0.0
74         augmented_x = np.vstack([zz, penalty])
75         augmented_y = np.concatenate([y, np.zeros(zz.shape[1])])
76         self.beta = np.linalg.lstsq(augmented_x, augmented_y, rcond=None)[0]
77
78     def predict(self, frame: pd.DataFrame) -> np.ndarray:
79         if self.mean is None or self.std is None or self.beta is None:
80             raise RuntimeError("model not fitted")
81         x = self.design(frame)
82         z = (x - self.mean) / self.std
83         zz = np.column_stack([np.ones(len(z)), z])
84         return np.einsum("ij,j->i", zz, self.beta)
85
86
87     def condition_profiles(df: pd.DataFrame) -> pd.DataFrame:
88         train = df[df["split"] == "train"].copy()
89         overall_load = train["dust_load_kg_h"].median()
90         labels = {
91             1: "低温低浓度高流量",
92             2: "高温中浓度低流量",
93             3: "高温高浓度高流量",
94             4: "低温高浓度低流量",
95         }
96         rows: list[dict[str, float | int | str]] = []
97         for cluster, group in train.groupby("condition_cluster"):
98             row: dict[str, float | int | str] = {
99                 "condition_cluster": int(cluster),
100                 "condition_name": labels[int(cluster)],
101                 "count": int(len(group)),
102                 "share": float(len(group) / len(train)),
103                 "Temp_C": float(group["Temp_C"].mean()),
104                 "C_in_gNm3": float(group["C_in_gNm3"].mean()),
105                 "Q_Nm3h": float(group["Q_Nm3h"].mean()),
106                 "dust_load_kg_h": float(group["dust_load_kg_h"].mean()),
107                 "C_out_recorded_median": float(group["C_out_mgNm3"].dropna().median()),
108                 "C_out_scaled_anchor": float((0.1 * group["C_out_mgNm3"].dropna()).median()),
109                 "historical_power_kw": float(group["P_total_kw"].mean()),
110             }
111             for col in U_COLS + T_COLS:
112                 row[f"{col}_p05"] = float(group[col].quantile(0.05))
113                 row[f"{col}_median"] = float(group[col].median())
114                 row[f"{col}_p95"] = float(group[col].quantile(0.95))
115             row["load_ratio_to_train_median"] = float(row["dust_load_kg_h"] / overall_load)
116             rows.append(row)
117         return pd.DataFrame(rows).sort_values("condition_cluster").reset_index(drop=True)

```



```

118
119
120 def scenario_t_star(
121     profile: pd.Series,
122     global_t_median: np.ndarray,
123     load_exponent: float = -0.18,
124 ) -> np.ndarray:
125     load_ratio = float(profile["load_ratio_to_train_median"])
126     raw = global_t_median * load_ratio ** load_exponent
127     lower = np.array([profile[f"{c}_p05"] for c in T_COLS], float)
128     upper = np.array([profile[f"{c}_p95"] for c in T_COLS], float)
129     return np.clip(raw, lower, upper)
130
131
132 def scenario_emission(
133     profile: pd.Series,
134     u: np.ndarray,
135     t: np.ndarray,
136     t_star: np.ndarray,
137     eta: float = 1.0,
138     peak_scale: float = 1.2,
139     safety_index: float = 0.08,
140     outlet_scale: float = 0.10,
141     alpha_weights: np.ndarray | None = None,
142     peak_exponent: float = 1.35,
143     phase_scale: float = 1.0,
144 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
145     """Return conservative base emission, rapping peak excess, and their sum.
146
147     All non-control arguments are scenario inputs,
148     not parameters estimated from the supplied outlet-concentration series.
149     """
150     u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
151     t_ref = np.array([profile[f"{c}_median"] for c in T_COLS], float)
152     cin = float(profile["C_in_gNm3"])
153     anchor = float(profile["C_out_recorded_median"]) * outlet_scale
154     k_anchor = math.log(cin * 1000.0 / anchor)
155
156     if alpha_weights is None:
157         alpha_weights = np.array([1.15, 1.10, 0.95, 0.80])
158     alpha = eta * np.asarray(alpha_weights, float)
159     control_delta = np.sum(alpha * ((u / u_ref) ** 2 - 1.0), axis=1)
160
161     load_ratio = float(profile["load_ratio_to_train_median"])
162     beta = np.array([0.30, 0.27, 0.20, 0.17]) * load_ratio ** 0.5
163
164     def loss(period: np.ndarray) -> np.ndarray:
165         ratio = period / t_star
166         return np.sum(beta * (ratio + 1.0 / ratio - 2.0), axis=1)
167
168     ref_loss = float(loss(t_ref.reshape(1, 4))[0])
169     k = k_anchor + control_delta - (loss(t) - ref_loss)
170     base = cin * 1000.0 * np.exp(-k + safety_index)
171
172     shares = np.array([0.30, 0.30, 0.22, 0.18])
173     peak = (
174         peak_scale
175         * phase_scale
176         * load_ratio ** 0.80
177         * np.sum(shares * (t / t_star) ** peak_exponent, axis=1)
178     )
179     return base, peak, base + peak

```

```

180
181
182 def support_statistics(train_group: pd.DataFrame) -> tuple[np.ndarray, np.ndarray, float]:
183     """Return the empirical control-support centre, inverse covariance and d2 cut-off."""
184     hist = train_group[U_COLS + T_COLS].to_numpy(float)
185     mu = hist.mean(axis=0)
186     inv_cov = np.linalg.pinv(np.cov(hist, rowvar=False))
187     delta = hist - mu
188     historical_d2 = np.einsum("ij,jk,ik->i", delta, inv_cov, delta)
189     return mu, inv_cov, float(np.quantile(historical_d2, 0.975))
190
191
192 def filter_supported_controls(
193     u: np.ndarray,
194     t: np.ndarray,
195     train_group: pd.DataFrame,
196 ) -> tuple[np.ndarray, np.ndarray, np.ndarray, float]:
197     mu, inv_cov, threshold = support_statistics(train_group)
198     controls = np.column_stack([u, t])
199     delta = controls - mu
200     mahal = np.einsum("ij,jk,ik->i", delta, inv_cov, delta)
201     ordering = (
202         (u[:, 0] >= u[:, 2] + 3.0)
203         & (u[:, 1] >= u[:, 3] + 3.0)
204         & (t[:, 2] >= t[:, 0] + 100.0)
205         & (t[:, 3] >= t[:, 1] + 100.0)
206     )
207     keep = (mahal <= threshold) & ordering
208     return u[keep], t[keep], mahal[keep], threshold
209
210
211 def candidate_bank(
212     profile: pd.Series,
213     model: RidgePowerModel,
214     t_star: np.ndarray,
215     n: int,
216     rng: np.random.Generator,
217     train_group: pd.DataFrame,
218 ) -> pd.DataFrame:
219     lower_u = np.array([profile[f"{c}_p05"] for c in U_COLS], float)
220     upper_u = np.array([profile[f"{c}_p95"] for c in U_COLS], float)
221     lower_t = np.array([profile[f"{c}_p05"] for c in T_COLS], float)
222     upper_t = np.array([profile[f"{c}_p95"] for c in T_COLS], float)
223
224     u = np.round(rng.uniform(lower_u, upper_u, size=(n, 4)), 1)
225     t = np.round(rng.uniform(lower_t, upper_t, size=(n, 4)))
226
227     # Ensure reference and locally useful candidates are always present.
228     ref_u = np.array([profile[f"{c}_median"] for c in U_COLS], float)
229     special_u = np.vstack([ref_u, lower_u, upper_u, 0.5 * (ref_u + lower_u), 0.5 * (ref_u + upper_u)])
230     special_t = np.vstack([
231         np.array([profile[f"{c}_median"] for c in T_COLS], float),
232         lower_t,
233         upper_t,
234         t_star,
235         np.clip(0.92 * t_star, lower_t, upper_t),
236     ])
237     u[: len(special_u)] = np.round(special_u, 1)
238     t[: len(special_t)] = np.round(special_t)
239
240     u, t, mahal, threshold = filter_supported_controls(u, t, train_group)
241

```

```

242     frame = pd.DataFrame(np.column_stack([u, t]), columns=U_COLS + T_COLS)
243     for c in COND_COLS:
244         frame[c] = float(profile[c])
245     frame["predicted_power_kW"] = model.predict(frame)
246     frame["mahalanobis_d2"] = mahal
247     frame["support_threshold_d2"] = threshold
248     return frame
249
250
251 def local_candidate_bank(
252     profile: pd.Series,
253     model: RidgePowerModel,
254     centre: dict[str, float | int | str],
255     n: int,
256     rng: np.random.Generator,
257     train_group: pd.DataFrame,
258 ) -> pd.DataFrame:
259     """Generate a clipped Gaussian neighbourhood around a preliminary solution."""
260     lower_u = np.array([profile[f"{c}_p05"] for c in U_COLS], float)
261     upper_u = np.array([profile[f"{c}_p95"] for c in U_COLS], float)
262     lower_t = np.array([profile[f"{c}_p05"] for c in T_COLS], float)
263     upper_t = np.array([profile[f"{c}_p95"] for c in T_COLS], float)
264     centre_u = np.array([centre[c] for c in U_COLS], float)
265     centre_t = np.array([centre[c] for c in T_COLS], float)
266     u = np.clip(
267         rng.normal(centre_u, np.maximum(0.15, 0.025 * (upper_u - lower_u)), size=(n, 4)),
268         lower_u,
269         upper_u,
270     )
271     t = np.clip(
272         rng.normal(centre_t, np.maximum(2.0, 0.025 * (upper_t - lower_t)), size=(n, 4)),
273         lower_t,
274         upper_t,
275     )
276     u = np.round(u, 1)
277     t = np.round(t)
278     u[0] = centre_u
279     t[0] = centre_t
280     u, t, mahal, threshold = filter_supported_controls(u, t, train_group)
281     frame = pd.DataFrame(np.column_stack([u, t]), columns=U_COLS + T_COLS)
282     for c in COND_COLS:
283         frame[c] = float(profile[c])
284     frame["predicted_power_kW"] = model.predict(frame)
285     frame["mahalanobis_d2"] = mahal
286     frame["support_threshold_d2"] = threshold
287     return frame
288
289
290 def select_optimum(
291     profile: pd.Series,
292     bank: pd.DataFrame,
293     t_star: np.ndarray,
294     limit: float,
295     eta: float = 1.0,
296     peak_scale: float = 1.2,
297     safety_index: float = 0.08,
298     outlet_scale: float = 0.10,
299     alpha_weights: np.ndarray | None = None,
300     peak_exponent: float = 1.35,
301     phase_scale: float = 1.0,
302     validation_error_guard_kW: float = 0.0,
303 ) -> dict[str, float | int | str]:

```

```

304 u = bank[U_COLS].to_numpy(float)
305 t = bank[T_COLS].to_numpy(float)
306 base, peak, total = scenario_emission(
307     profile,
308     u,
309     t,
310     t_star,
311     eta,
312     peak_scale,
313     safety_index,
314     outlet_scale,
315     alpha_weights,
316     peak_exponent,
317     phase_scale,
318 )
319 feasible = total <= limit
320 if not np.any(feasible):
321     return {
322         "condition_cluster": int(profile["condition_cluster"]),
323         "condition_name": str(profile["condition_name"]),
324         "limit_mgNm3": float(limit),
325         "status": "INFEASIBLE_IN_SCENARIO_SUPPORT",
326     }
327 idx_pool = np.flatnonzero(feasible)
328 local = bank["predicted_power_kw"].to_numpy(float)[idx_pool]
329 idx = int(idx_pool[int(np.argmax(local))])
330 row: dict[str, float | int | str] = {
331     "condition_cluster": int(profile["condition_cluster"]),
332     "condition_name": str(profile["condition_name"]),
333     "limit_mgNm3": float(limit),
334     "status": "SCENARIO_FEASIBLE",
335     "scenario_base_emission_mgNm3": float(base[idx]),
336     "scenario_peak_excess_mgNm3": float(peak[idx]),
337     "scenario_peak_total_mgNm3": float(total[idx]),
338     "predicted_power_kw": float(bank.iloc[idx]["predicted_power_kw"]),
339     "validation_error_adjusted_power_kw": float(
340         bank.iloc[idx]["predicted_power_kw"] + validation_error_guard_kw
341     ),
342     "validation_error_guard_kw": float(validation_error_guard_kw),
343     "historical_power_kw": float(profile["historical_power_kw"]),
344     "power_change_vs_historical_pct": float(
345         100.0 * (bank.iloc[idx]["predicted_power_kw"] / profile["historical_power_kw"] - 1.0)
346     ),
347     "mahalanobis_d2": float(bank.iloc[idx]["mahalanobis_d2"]),
348     "support_threshold_d2": float(bank.iloc[idx]["support_threshold_d2"]),
349 }
350 for col in U_COLS + T_COLS:
351     row[col] = float(bank.iloc[idx][col])
352 return row
353
354
355 def make_figures(
356     df: pd.DataFrame,
357     power_model: RidgePowerModel,
358     profiles: pd.DataFrame,
359     optimum: pd.DataFrame,
360     sensitivity: pd.DataFrame,
361 ) -> None:
362     plt.rcParams.update({
363         "font.sans-serif": ["Arial Unicode MS", "PingFang SC", "Heiti TC", "DejaVu Sans"],
364         "axes.unicode_minus": False,
365         "figure.dpi": 160,

```

```

366     "savefig.dpi": 240,
367 })
368
369 colors = ["#0072B2", "#E69F00", "#009E73", "#CC79A7"]
370 train = df[df["split"] == "train"]
371 fig, ax = plt.subplots(figsize=(8.4, 5.4))
372 for (cluster, group), color in zip(train.groupby("condition_cluster"), colors):
373     sample = group.iloc[:4]
374     ax.scatter(sample["C_in_gNm3"], sample["Temp_C"], s=7, alpha=0.35, color=color,
375               label=f"工况{int(cluster)}")
376 ax.set_xlabel("入口粉尘浓度 (g/Nm³)")
377 ax.set_ylabel("入口温度 (°C)")
378 ax.set_title("训练期典型工况划分")
379 ax.legend(ncol=2, frameon=False)
380 fig.tight_layout()
381 fig.savefig(FIG_DIR / "01_condition_clusters.png")
382 plt.close(fig)
383
384 test = df[df["split"] == "test"]
385 pred = power_model.predict(test)
386 actual = test["P_total_kw"].to_numpy(float)
387 fig, ax = plt.subplots(figsize=(6.2, 5.4))
388 ax.scatter(actual, pred, s=9, alpha=0.45, color="#0072B2")
389 lo = min(actual.min(), pred.min())
390 hi = max(actual.max(), pred.max())
391 ax.plot([lo, hi], [lo, hi], "--", color="#D55E00", linewidth=1.3)
392 ax.set_xlabel("实际总功率 (kW)")
393 ax.set_ylabel("预测总功率 (kW)")
394 ax.set_title("功率模型回顾性测试")
395 fig.tight_layout()
396 fig.savefig(FIG_DIR / "02_power_test.png")
397 plt.close(fig)
398
399 pivot = optimum.pivot(index="condition_name", columns="limit_mgNm3", values="predicted_power_kw")
400 pivot = pivot[[10.0, 5.0]]
401 fig, ax = plt.subplots(figsize=(9.2, 5.4))
402 x = np.arange(len(pivot))
403 width = 0.34
404 ax.bar(x - width / 2, pivot[10.0], width, label="10 mg/Nm³", color="#56B4E9")
405 ax.bar(x + width / 2, pivot[5.0], width, label="5 mg/Nm³", color="#D55E00")
406 ax.set_xticks(x, pivot.index, rotation=12)
407 ax.set_ylabel("情景最优功率 (kW)")
408 ax.set_title("不同排放约束下的情景最优功率")
409 ax.legend(frameon=False)
410 fig.tight_layout()
411 fig.savefig(FIG_DIR / "03_optimal_power_comparison.png")
412 plt.close(fig)
413
414 fig, ax = plt.subplots(figsize=(7.6, 5.1))
415 ratio = np.linspace(0.75, 1.25, 200)
416 for profile, color in zip((profiles.iloc[0], profiles.iloc[2]), (colors[0], colors[2])):
417     load_ratio = float(profile["load_ratio_to_train_median"])
418     peak = 1.2 * load_ratio ** 0.8 * ratio ** 1.35
419     ax.plot(100 * (ratio - 1), peak, color=color, linewidth=2,
420           label=str(profile["condition_name"]))
421 ax.set_xlabel("振打周期相对最优周期的变化 (%)")
422 ax.set_ylabel("单次振打峰值增量代理 (mg/Nm³)")
423 ax.set_title("振打周期延长会放大单次再飞扬峰值")
424 ax.axvline(0, color="#666666", linewidth=0.8)
425 ax.legend(frameon=False)
426 fig.tight_layout()
427 fig.savefig(FIG_DIR / "04_rapping_peak_scenario.png")

```

```

428 plt.close(fig)
429
430 fig, ax = plt.subplots(figsize=(7.6, 5.1))
431 vals = sensitivity["weighted_power_increase_pct"].dropna().sort_values().to_numpy()
432 ax.hist(vals, bins=12, color="#009E73", alpha=0.85, edgecolor="white")
433 ax.axvline(np.median(vals), color="#D55E00", linestyle="--", linewidth=1.5,
434            label=f"中位数 {np.median(vals):.1f}%")
435 ax.set_xlabel("5 mg/Nm³ 相对 10 mg/Nm³ 的加权功率增幅 (%)")
436 ax.set_ylabel("情景组合数")
437 ax.set_title("先验参数敏感性分析")
438 ax.legend(frameon=False)
439 fig.tight_layout()
440 fig.savefig(FIG_DIR / "05_sensitivity_distribution.png")
441 plt.close(fig)
442
443
444 def evaluate_power_models(
445     train: pd.DataFrame,
446     validation: pd.DataFrame,
447     test: pd.DataFrame,
448 ) -> tuple(RidgePowerModel, dict[str, object], pd.DataFrame, pd.DataFrame):
449     """Select ridge strength by blocked rolling validation and retain honest baselines."""
450     alpha_grid = (1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0)
451     n = len(train)
452     rolling_bounds = ((0, int(0.40*n), int(0.60*n)), (0, int(0.60*n), int(0.80*n)), (0, int(0.80*n), n))
453     rolling_rows: list[dict[str, object]] = []
454     selected: dict[str, float] = {}
455     for model_name, quadratic in (("linear_ridge", False), ("quadratic_ridge", True)):
456         alpha_scores: list[tuple[float, float]] = []
457         for alpha in alpha_grid:
458             fold_rmse = []
459             for fold, (start, fit_end, eval_end) in enumerate(rolling_bounds, start=1):
460                 fit = train.iloc[start:fit_end]
461                 evaluate = train.iloc[fit_end:eval_end]
462                 candidate = RidgePowerModel(alpha=alpha, quadratic=quadratic)
463                 candidate.fit(fit, fit["P_total_kw"].to_numpy(float))
464                 pred = candidate.predict(evaluate)
465                 actual = evaluate["P_total_kw"].to_numpy(float)
466                 rmse = _rmse(actual, pred)
467                 fold_rmse.append(rmse)
468                 rolling_rows.append({
469                     "model": model_name,
470                     "alpha": alpha,
471                     "fold": fold,
472                     "fit_rows": len(fit),
473                     "evaluation_rows": len(evaluate),
474                     "r2": _r2(actual, pred),
475                     "rmse_kw": rmse,
476                     "bias_kw": float(np.mean(pred-actual)),
477                 })
478             alpha_scores.append((float(np.mean(fold_rmse)), alpha))
479     selected[model_name] = min(alpha_scores)[1]
480
481     baseline_rows: list[dict[str, object]] = []
482     mean_power = float(train["P_total_kw"].mean())
483     fitted_models: dict[str, RidgePowerModel] = {}
484     for model_name, quadratic in (("linear_ridge", False), ("quadratic_ridge", True)):
485         fitted = RidgePowerModel(alpha=selected[model_name], quadratic=quadratic)
486         fitted.fit(train, train["P_total_kw"].to_numpy(float))
487         fitted_models[model_name] = fitted
488     for split_name, frame in (("validation", validation), ("retrospective_test", test)):
489         actual = frame["P_total_kw"].to_numpy(float)

```

```

490     for model_name in ("mean_predictor", "linear_ridge", "quadratic_ridge"):
491         pred = np.full((len(frame), mean_power) if model_name == "mean_predictor" else fitted_models[model_name].predict(frame)
492         baseline_rows.append({
493             "model": model_name,
494             "selected_alpha": np.nan if model_name == "mean_predictor" else selected[model_name],
495             "split": split_name,
496             "r2": _r2(actual, pred),
497             "rmse_kw": _rmse(actual, pred),
498             "bias_kw": float(np.mean(pred-actual)),
499         })
500     chosen = fitted_models["quadratic_ridge"]
501     val_actual = validation["P_total_kw"].to_numpy(float)
502     val_pred = chosen.predict(validation)
503     test_actual = test["P_total_kw"].to_numpy(float)
504     test_pred = chosen.predict(test)
505     guard = float(np.quantile(np.abs(val_pred-val_actual), 0.90))
506     metrics: dict[str, object] = {
507         "model": "standardized_quadratic_ridge_voltage_plus_period_and_condition",
508         "fit_scope": "train_only",
509         "alpha_selected_by_blocked_rolling_validation": selected["quadratic_ridge"],
510         "validation_r2": _r2(val_actual, val_pred),
511         "validation_rmse_kw": _rmse(val_actual, val_pred),
512         "validation_bias_kw": float(np.mean(val_pred-val_actual)),
513         "validation_abs_error_q90_kw": guard,
514         "retrospective_test_r2": _r2(test_actual, test_pred),
515         "retrospective_test_rmse_kw": _rmse(test_actual, test_pred),
516         "retrospective_test_bias_kw": float(np.mean(test_pred-test_actual)),
517         "test_protocol": "retrospective_not_blind_due_to_prior_package_use",
518         "reporting_guard_rule": "point_prediction_plus_validation_absolute_error_q90",
519         "selected_alphas": selected,
520     }
521     return chosen, metrics, pd.DataFrame(rolling_rows), pd.DataFrame(baseline_rows)
522
523
524 def aggregate_pair(rows10: List[dict[str, object]], rows5: List[dict[str, object]]) -> dict[str, float | bool]:
525     if len(rows10) != 4 or len(rows5) != 4:
526         return {"all_conditions_feasible": False, "weighted_power_10_kw": np.nan, "weighted_power_5_kw": np.nan, "weighted_power_increase_pct":
527             ↪ "": np.nan}
528     weights = np.array([float(r["share"]) for r in rows10])
529     p10 = float(np.average([float(r["predicted_power_kw"]) for r in rows10], weights=weights))
530     p5 = float(np.average([float(r["predicted_power_kw"]) for r in rows5], weights=weights))
531     return {"all_conditions_feasible": True, "weighted_power_10_kw": p10, "weighted_power_5_kw": p5, "weighted_power_increase_pct": 100.0*(p5
532             ↪ /p10-1.0)}
533
534 def main() -> None:
535     OUT_DIR.mkdir(parents=True, exist_ok=True)
536     FIG_DIR.mkdir(parents=True, exist_ok=True)
537     df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
538     df["condition_cluster"] = df["condition_cluster"].astype(int)
539     profiles = condition_profiles(df)
540     profiles.to_csv(OUT_DIR / "condition_profiles.csv", index=False, encoding="utf-8-sig")
541
542     train = df[df["split"] == "train"].copy()
543     validation = df[df["split"] == "validation"].copy()
544     test = df[df["split"] == "test"].copy()
545     model, metrics, rolling_metrics, baseline_metrics = evaluate_power_models(train, validation, test)
546     rolling_metrics.to_csv(OUT_DIR / "power_model_rolling_validation.csv", index=False, encoding="utf-8-sig")
547     baseline_metrics.to_csv(OUT_DIR / "power_model_baselines.csv", index=False, encoding="utf-8-sig")
548     (OUT_DIR / "power_model_metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8")
549     validation_error_guard = float(metrics["validation_abs_error_q90_kw"])

```

```

550 cluster_eval = pd.read_csv(CLUSTER_EVAL_FILE)
551 cluster_eval.to_csv(OUT_DIR / "cluster_selection_metrics.csv", index=False, encoding="utf-8-sig")
552 global_t_median = train[T_COLS].median().to_numpy(float)
553 t_stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median) for _, p in profiles.iterrows()}
554
555 banks: dict[int, pd.DataFrame] = {}
556 seed_rows: list[dict[str, object]] = []
557 convergence_rows: list[dict[str, object]] = []
558 for seed in SEEDS:
559     rng = np.random.default_rng(seed)
560     for _, profile in profiles.iterrows():
561         cluster = int(profile["condition_cluster"])
562         group = train[train["condition_cluster"] == cluster]
563         t_star = t_stars[cluster]
564         base_bank = candidate_bank(profile, model, t_star, 220_000, rng, group)
565         preliminary = {
566             limit: select_optimum(
567                 profile,
568                 base_bank,
569                 t_star,
570                 limit,
571                 validation_error_guard_kw=validation_error_guard,
572             )
573             for limit in (10.0, 5.0)
574         }
575         additions = []
576         for limit in (10.0, 5.0):
577             if "predicted_power_kw" in preliminary[limit]:
578                 additions.append(local_candidate_bank(profile, model, preliminary[limit], 30_000, rng, group))
579         bank = pd.concat([base_bank] + additions, ignore_index=True).drop_duplicates(U_COLS+T_COLS, keep="first")
580         if seed == SEED:
581             banks[cluster] = bank
582             for limit in (10.0, 5.0):
583                 for fraction in (0.25, 0.50, 0.75, 1.00):
584                     partial = base_bank.iloc[:max(1000, int(len(base_bank)*fraction))]
585                     result = select_optimum(
586                         profile,
587                         partial,
588                         t_star,
589                         limit,
590                         validation_error_guard_kw=validation_error_guard,
591                     )
592                     convergence_rows.append({
593                         "condition_cluster": cluster,
594                         "condition_name": profile["condition_name"],
595                         "limit_mgNm3": limit,
596                         "candidate_bank_fraction": fraction,
597                         "supported_candidates": len(partial),
598                         "status": result["status"],
599                         "best_power_kw": result.get("predicted_power_kw", np.nan),
600                     })
601             for limit in (10.0, 5.0):
602                 final = select_optimum(
603                     profile,
604                     bank,
605                     t_star,
606                     limit,
607                     validation_error_guard_kw=validation_error_guard,
608                 )
609                 final.update({
610                     "seed": seed,
611                     "base_supported_candidates": len(base_bank),

```



```

612         "refined_supported_candidates": len(bank),
613         "preliminary_power_kw": preliminary[limit].get("predicted_power_kw", np.nan),
614         "local_refinement_improvement_pct": 100.0*(float(preliminary[limit].get("predicted_power_kw", np.nan))/float(final.get("
↳ predicted_power_kw", np.nan))-1.0),
615     })
616     seed_rows.append(final)
617
618 seed_stability = pd.DataFrame(seed_rows)
619 seed_stability.to_csv(OUT_DIR / "optimization_seed_stability.csv", index=False, encoding="utf-8-sig")
620 central = seed_stability[seed_stability["seed"] == SEED].copy()
621 optimum = central.drop(columns=["seed", "base_supported_candidates", "refined_supported_candidates", "preliminary_power_kw", "
↳ local_refinement_improvement_pct"])
622 optimum.to_csv(OUT_DIR / "optimal_controls_central_scenario.csv", index=False, encoding="utf-8-sig")
623 pd.DataFrame(convergence_rows).to_csv(OUT_DIR / "optimization_convergence.csv", index=False, encoding="utf-8-sig")
624
625 q4_rows: list[dict[str, object]] = []
626 for _, profile in profiles.iterrows():
627     cluster = int(profile["condition_cluster"])
628     a = optimum[(optimum["condition_cluster"] == cluster) & (optimum["limit_mgNm3"] == 10.0)].iloc[0]
629     b = optimum[(optimum["condition_cluster"] == cluster) & (optimum["limit_mgNm3"] == 5.0)].iloc[0]
630     q4_rows.append({
631         "condition_cluster": cluster,
632         "condition_name": profile["condition_name"],
633         "share": profile["share"],
634         "power_10_kw": a["predicted_power_kw"],
635         "power_5_kw": b["predicted_power_kw"],
636         "validation_error_adjusted_power_10_kw": a["validation_error_adjusted_power_kw"],
637         "validation_error_adjusted_power_5_kw": b["validation_error_adjusted_power_kw"],
638         "increase_pct": 100.0*(b["predicted_power_kw"]/a["predicted_power_kw"]-1.0),
639     })
640 q4 = pd.DataFrame(q4_rows)
641 weighted_p10 = float(np.average(q4["power_10_kw"], weights=q4["share"]))
642 weighted_p5 = float(np.average(q4["power_5_kw"], weights=q4["share"]))
643 q4_summary = {
644     "weighted_power_10_kw": weighted_p10,
645     "weighted_power_5_kw": weighted_p5,
646     "weighted_increase_pct": 100.0*(weighted_p5/weighted_p10-1.0),
647     "validation_error_guard_kw": validation_error_guard,
648     "weighted_validation_error_adjusted_power_10_kw": weighted_p10 + validation_error_guard,
649     "weighted_validation_error_adjusted_power_5_kw": weighted_p5 + validation_error_guard,
650 }
651 q4.to_csv(OUT_DIR / "question4_by_condition.csv", index=False, encoding="utf-8-sig")
652 (OUT_DIR / "question4_summary.json").write_text(json.dumps(q4_summary, ensure_ascii=False, indent=2), encoding="utf-8")
653
654 sensitivity_rows: list[dict[str, object]] = []
655 for eta in (0.8, 1.0, 1.2):
656     for peak_scale in (0.9, 1.2, 1.5):
657         for safety in (0.0, 0.08, 0.15):
658             r10s: list[dict[str, object]] = []
659             r5s: list[dict[str, object]] = []
660             for _, profile in profiles.iterrows():
661                 cluster = int(profile["condition_cluster"])
662                 r10 = select_optimum(profile, banks[cluster], t_stars[cluster], 10.0, eta=eta, peak_scale=peak_scale, safety_index=safety
↳ , validation_error_guard_kw=validation_error_guard)
663                 r5 = select_optimum(profile, banks[cluster], t_stars[cluster], 5.0, eta=eta, peak_scale=peak_scale, safety_index=safety,
↳ validation_error_guard_kw=validation_error_guard)
664                 if "predicted_power_kw" in r10 and "predicted_power_kw" in r5:
665                     r10["share"] = profile["share"]
666                     r5["share"] = profile["share"]
667                     r10s.append(r10); r5s.append(r5)
668             sensitivity_rows.append({"eta_voltage_effect": eta, "peak_scale": peak_scale, "safety_index": safety, **aggregate_pair(r10s,
↳ r5s)})

```

```

669 sensitivity = pd.DataFrame(sensitivity_rows)
670 sensitivity.to_csv(OUT_DIR / "question4_sensitivity.csv", index=False, encoding="utf-8-sig")
671
672 alpha_profiles = {
673     "equal": np.array([1.00, 1.00, 1.00, 1.00]),
674     "weak_front": np.array([1.08, 1.04, 0.98, 0.90]),
675     "central_front": np.array([1.15, 1.10, 0.95, 0.80]),
676 }
677 structural_rows: list[dict[str, object]] = []
678 for outlet_scale in (0.08, 0.09, 0.10, 0.11, 0.12):
679     for alpha_name, alpha_weights in alpha_profiles.items():
680         for peak_exponent in (1.10, 1.35, 1.60):
681             for phase_scale in (0.65, 0.80, 1.00):
682                 for tstar_exponent in (-0.30, -0.18, -0.10):
683                     r10s = []; r5s = []
684                     for _, profile in profiles.iterrows():
685                         cluster = int(profile["condition_cluster"])
686                         t_star = scenario_t_star(profile, global_t_median, tstar_exponent)
687                         args = dict(outlet_scale=outlet_scale, alpha_weights=alpha_weights, peak_exponent=peak_exponent, phase_scale=
↪ phase_scale, validation_error_guard_kw=validation_error_guard)
688                         r10 = select_optimum(profile, banks[cluster], t_star, 10.0, **args)
689                         r5 = select_optimum(profile, banks[cluster], t_star, 5.0, **args)
690                         if "predicted_power_kw" in r10 and "predicted_power_kw" in r5:
691                             r10["share"] = profile["share"]; r5["share"] = profile["share"]
692                             r10s.append(r10); r5s.append(r5)
693                     structural_rows.append({
694                         "outlet_scale": outlet_scale,
695                         "alpha_profile": alpha_name,
696                         "peak_exponent": peak_exponent,
697                         "phase_scale": phase_scale,
698                         "tstar_load_exponent": tstar_exponent,
699                         **aggregate_pair(r10s, r5s),
700                     })
701 structural = pd.DataFrame(structural_rows)
702 structural.to_csv(OUT_DIR / "structural_sensitivity.csv", index=False, encoding="utf-8-sig")
703
704 ablation_rows: list[dict[str, object]] = []
705 ablation_summary_rows: list[dict[str, object]] = []
706 for alpha_name, alpha_weights in alpha_profiles.items():
707     per_profile = []
708     for _, profile in profiles.iterrows():
709         cluster = int(profile["condition_cluster"])
710         args = dict(alpha_weights=alpha_weights, validation_error_guard_kw=validation_error_guard)
711         r10 = select_optimum(profile, banks[cluster], t_stars[cluster], 10.0, **args)
712         r5 = select_optimum(profile, banks[cluster], t_stars[cluster], 5.0, **args)
713         row = {"alpha_profile": alpha_name, "condition_cluster": cluster, "condition_name": profile["condition_name"], "share": profile["
↪ share"]}
714         for i, col in enumerate(U_COLS, start=1):
715             row[f"delta_U{i}_kV"] = float(r5[col]) - float(r10[col])
716         per_profile.append(row); ablation_rows.append(row)
717     weights = np.array([float(x["share"]) for x in per_profile])
718     deltas = np.array([float(x[f"delta_U{i}_kV"]) for i in range(1,5)] for x in per_profile])
719     weighted = np.average(deltas, axis=0, weights=weights)
720     positive = np.maximum(weighted, 0)
721     ablation_summary_rows.append({
722         "alpha_profile": alpha_name,
723         **{f"weighted_delta_U{i}_kV": weighted[i-1] for i in range(1,5)},
724         "front_two_share_of_positive_adjustment": float(positive[:2].sum()/positive.sum()) if positive.sum() else np.nan,
725     })
726 pd.DataFrame(ablation_rows).to_csv(OUT_DIR / "field_priority_ablation_by_condition.csv", index=False, encoding="utf-8-sig")
727 ablation_summary = pd.DataFrame(ablation_summary_rows)
728 ablation_summary.to_csv(OUT_DIR / "field_priority_ablation_summary.csv", index=False, encoding="utf-8-sig")

```

```

729
730 historical_rows: list[dict[str, object]] = []
731 dynamic_rows: list[dict[str, object]] = []
732 for _, profile in profiles.iterrows():
733     cluster = int(profile["condition_cluster"])
734     frame = pd.DataFrame([**{c: profile[f"{c}_median"] for c in U_COLS+T_COLS}, **{c: profile[c] for c in COND_COLS}])
735     median_power = float(model.predict(frame)[0])
736     r10 = optimum[(optimum["condition_cluster"] == cluster) & (optimum["Limit_mgNm3"] == 10.0)].iloc[0]
737     r5 = optimum[(optimum["condition_cluster"] == cluster) & (optimum["Limit_mgNm3"] == 5.0)].iloc[0]
738     historical_rows.append({
739         "condition_cluster": cluster,
740         "condition_name": profile["condition_name"],
741         "share": profile["share"],
742         "historical_mean_actual_power_kw": profile["historical_power_kw"],
743         "model_power_at_historical_median_controls_kw": median_power,
744         "scenario_optimal_10_power_kw": r10["predicted_power_kw"],
745         "auxiliary_saving_vs_model_median_pct": 100.0*(median_power/float(r10["predicted_power_kw"])-1.0),
746     })
747     dynamic_rows.append({
748         "condition_cluster": cluster,
749         "condition_name": profile["condition_name"],
750         "mode_selection_rule": "EXTERNAL_TARGET_SELECTION",
751         "available_modes": "TARGET-10/TARGET-5/FALLBACK",
752         "automatic_emission_threshold_switching": False,
753         "provisional_voltage_ramp_kV_per_min": 0.5,
754         "provisional_period_ramp_s_per_min": 5.0,
755         "phase_scenario_for_online_trial": "0.65/0.80/1.00",
756         "implementation_status": "PROVISIONAL_MUST_BE_OVERRIDDEN_BY_SITE_DCS_AND_INTERLOCK_LIMITS",
757         **{f"target10_{c}": r10[c] for c in U_COLS+T_COLS},
758         **{f"target5_{c}": r5[c] for c in U_COLS+T_COLS},
759     })
760 pd.DataFrame(historical_rows).to_csv(OUT_DIR / "historical_median_power_comparison.csv", index=False, encoding="utf-8-sig")
761 pd.DataFrame(dynamic_rows).to_csv(OUT_DIR / "dynamic_control_schedule.csv", index=False, encoding="utf-8-sig")
762
763 feasible_structural = structural[structural["all_conditions_feasible"] == True]
764 scenario_params = {
765     "status": "SCENARIO_ASSUMPTIONS_NOT_IDENTIFIED_FROM_C_OUT",
766     "central_outlet_scale": 0.10,
767     "structural_outlet_scales": [0.08, 0.09, 0.10, 0.11, 0.12],
768     "voltage_weight_profiles": {k: v.tolist() for k, v in alpha_profiles.items()},
769     "peak_exponents": [1.10, 1.35, 1.60],
770     "rapping_phase_scales": [0.65, 0.80, 1.00],
771     "tstar_load_exponents": [-0.30, -0.18, -0.10],
772     "rapping_loss_coefficients": [0.30, 0.27, 0.20, 0.17],
773     "peak_field_shares": [0.30, 0.30, 0.22, 0.18],
774     "support_rule": "cluster-specific P05-P95 plus empirical 97.5th percentile Mahalanobis d2 and engineering ordering",
775     "base_candidate_count_per_condition_seed": 220000,
776     "local_candidate_count_per_preliminary_solution": 30000,
777     "random_seeds": SEEDS,
778 }
779 (OUT_DIR / "scenario_parameters.json").write_text(json.dumps(scenario_params, ensure_ascii=False, indent=2), encoding="utf-8")
780
781 make_figures(df, model, profiles, optimum, sensitivity)
782 seed_grouped = seed_stability.groupby(["condition_cluster", "Limit_mgNm3"])["predicted_power_kw"]
783 summary = {
784     "power_model": metrics,
785     "cluster_k": 4,
786     "central_q4": q4_summary,
787     "prior_parameter_sensitivity": {
788         "increase_pct_min": float(sensitivity["weighted_power_increase_pct"].min()),
789         "increase_pct_median": float(sensitivity["weighted_power_increase_pct"].median()),
790         "increase_pct_max": float(sensitivity["weighted_power_increase_pct"].max()),
791     }

```

```

791         "feasible_combinations": int(sensitivity["all_conditions_feasible"].sum()),
792         "total_combinations": len(sensitivity),
793     },
794     "structural_sensitivity": {
795         "feasible_combinations": len(feasible_structural),
796         "total_combinations": len(structural),
797         "increase_pct_min": float(feasible_structural["weighted_power_increase_pct"].min()),
798         "increase_pct_median": float(feasible_structural["weighted_power_increase_pct"].median()),
799         "increase_pct_max": float(feasible_structural["weighted_power_increase_pct"].max()),
800     },
801     "seed_stability_max_cv_pct": float((100.0*seed_grouped.std()/seed_grouped.mean()).max()),
802     "seed_stability_max_range_pct_of_mean": float((100.0*(seed_grouped.max()-seed_grouped.min())/seed_grouped.mean()).max()),
803     "local_refinement_max_improvement_pct": float(seed_stability["local_refinement_improvement_pct"].max()),
804     "central_all_conditions_feasible": bool((optimum["status"] == "SCENARIO_FEASIBLE").all()),
805 }
806 (OUT_DIR / "model_run_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
807 print(json.dumps(summary, ensure_ascii=False, indent=2))
808
809
810 if __name__ == "__main__":
811     main()

```

D.2 paper_figures.py

```

1  #!/usr/bin/env python3
2  """Generate the complete publication figure set for the cement ESP paper.
3
4  All empirical plots are regenerated from the supplied processed dataset and
5  model result tables. Scenario-only plots are explicitly labelled in captions.
6  Both PNG previews and vector PDF files are exported.
7  """
8
9  from __future__ import annotations
10
11  import sys
12  import hashlib
13  from pathlib import Path
14
15  import matplotlib.pyplot as plt
16  from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
17  import numpy as np
18  import pandas as pd
19
20  SRC_DIR = Path(__file__).resolve().parent
21  if str(SRC_DIR) not in sys.path:
22      sys.path.insert(0, str(SRC_DIR))
23
24  from plot_utils import GRAY, LIGHT_GRAY, PALETTE, add_subpanel_label, finish_axes, save_figure, set_paper_style
25  from scenario_model import RidgePowerModel, T_COLS, U_COLS
26
27
28  ROOT = SRC_DIR.parent
29  DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
30  RESULTS = ROOT / "results"
31  OUT = ROOT / "figures_paper"
32
33  COND_SHORT = {
34      1: "工况1\n低温低浓高流",
35      2: "工况2\n高温中浓低流",
36      3: "工况3\n高温高浓高流",
37      4: "工况4\n低温高浓低流",

```

```

38 }
39
40 CAPTIONS = {
41     "01_outlet_data_diagnosis": "出口浓度记录存在显著截顶且不覆盖目标区间。有效记录全部位于48.74~50.00 mg/Nm³，其中大量观测
    ⇨ 等于50 mg/Nm³，因而不能从该列直接辨识5或10 mg/Nm³附近的控制—排放关系。",
42     "02_process_timeseries": "原始过程量具有连续时序变化和多工况切换特征。图中展示首24小时入口温度、入口浓度、烟气流量及总
    ⇨ 功率的分钟级变化。",
43     "03_cluster_selection": "四类工况在轮廓系数与解释复杂度之间取得较好平衡。k=4的轮廓系数达到0.408，且避免继续增加聚类数带
    ⇨ 来的解释负担。",
44     "04_condition_map": "四类工况在入口浓度—温度平面上形成清晰分区，并由烟气流量大小补充刻画负荷差异。",
45     "05_condition_profile_heatmap": "标准化工况画像揭示温度、浓度、流量、粉尘负荷与历史总功率之间的结构差异，为分工况控制提
    ⇨ 供依据。",
46     "06_method_overview": "本文采用数据驱动功率模型与工程灰箱排放模型协同的分层优化框架。实测数据负责功率与工况辨识，排放
    ⇨ 约束仅在显式先验情景下推演。",
47     "07_power_prediction": "滚动验证选择的二次岭回归优于均值与线性基线；回顾性测试R²为0.957、RMSE为14.76 kW。左图虚线表示理想
    ⇨ 预测，右图比较两时段RMSE。",
48     "08_power_residuals": "回顾性测试残差均值为-10.78 kW，且在高功率区负偏差有所扩大；验证集绝对误差90%分位数11.78 kW作为验
    ⇨ 证期误差附加量。",
49     "08b_feature_importance": "二次岭功率模型的标准化系数显示，电压及其二次交互项是主要预测信息来源。系数反映相关性和预测
    ⇨ 贡献，不解释为单变量因果效应。",
50     "09_voltage_power_response": "在各工况典型状态下，模型给出的总功率随四电场电压同步提高而上升。曲线仅在各工况历史电压支
    ⇨ 持区间附近绘制。",
51     "10_optimal_power": "更严格的5 mg/Nm³情景约束在四类工况下均要求更高预测功率；中心情景工况加权平均功率由1609.42 kW升至1829
    ⇨ .79 kW。",
52     "11_optimal_voltage": "5 mg/Nm³情景主要通过提高前级电场电压并适度调整后级电压实现更高捕集强度。各柱为随机候选搜索得到
    ⇨ 的支持域内最优解。",
53     "12_optimal_rapping_period": "最优振打周期在不同工况和排放限值间发生协同调整，前两电场周期较短、后两电场周期较长的工程
    ⇨ 顺序约束始终得到保持。",
54     "12b_radar_controls": "代表工况的电压和振打周期剖面表明，限值收紧时前级电压增量更突出，但后级补偿与周期协同仍不可忽
    ⇨ 略。",
55     "13_emission_decomposition": "最优方案的约束值由连续排放基值和振打峰值增量共同构成。柱顶虚线分别表示10和5 mg/Nm³情景限
    ⇨ 值。",
56     "14_rapping_peak_mechanism": "灰箱情景表明，振打周期相对参考最优周期延长会放大单次再飞扬峰值；高粉尘负荷工况的峰值增幅
    ⇨ 更显著。",
57     "15_search_convergence": "五个独立随机种子与局部细化得到的最低预测功率高度稳定；八个工况—限值组合的最大变异系数为0
    ⇨ .181%，局部细化相对初始随机库最多改善0.986‰。",
58     "16_sensitivity_distribution": "405组结构情景全部在当前支持域内可行；5 mg/Nm³相对10 mg/Nm³的加权功率增幅范围为11.37%~14.31%
    ⇨ ，中位数为12.99%。箱线图展示出口缩放与振打相位的联合影响。",
59     "17_sensitivity_heatmap": "场权重消融后，前两电场仍承担70.8%~73.7%的正向电压增量，说明前级优先并非完全由中心权重先验写
    ⇨ 入，但后级电压仍有不可忽略的补偿作用。",
60     "18_condition_energy_penalty": "四类工况从10收紧至5 mg/Nm³的中心情景功率增幅为13.34%~14.11%，加权平均为13.69%；在相同运行时
    ⇨ 长下，该比例也对应电耗增幅。",
61     "05b_boundary_proximity": "八组推荐解均未超过各工况经验97.5%支持域边界，但工况2的10 mg解和工况4的5 mg解已超过90%贴近阈
    ⇨ 值，必须经现场阶跃试验复核。",
62 }
63
64 TRACE_META = {
65     "01_outlet_data_diagnosis": (str(DATA_FILE), "fig01_outlet_diagnosis", "§ 2.2；图1", "出口记录可能为量程截顶；不代表真实排放分
    ⇨ 布"),
66     "02_process_timeseries": (str(DATA_FILE), "fig02_timeseries", "§ 2.1—2.2；图2", "只展示首24 h，不代表全周期分布"),
67     "03_cluster_selection": (str(RESULTS / "cluster_selection_metrics.csv"), "fig03_cluster_selection", "§ 5.1；图5", "聚类指标只评价统计
    ⇨ 分离度，不等于物理工况唯一性"),
68     "04_condition_map": (str(DATA_FILE), "fig04_condition_map", "§ 5.1；图6", "为可视化抽样点；聚类实际使用全部训练样本"),
69     "05_condition_profile_heatmap": (str(RESULTS / "condition_profiles.csv"), "fig05_profile_heatmap", "§ 5.1；图7", "列内标准化仅用于相
    ⇨ 对比较"),
70     "06_method_overview": (str(ROOT / "10_修订后完整论文_核验版.md"), "fig06_method_overview", "§ 3；图3", "概念流程图，不含新的经
    ⇨ 验数据"),
71     "07_power_prediction": (str(RESULTS / "power_model_baselines.csv"), "fig07_power_prediction + RidgePowerModel", "§ 5.2；图8", "回顾性测
    ⇨ 试并非真正未触碰盲测"),
72     "08_power_residuals": (str(DATA_FILE), "fig08_residuals + RidgePowerModel", "§ 5.2；图10", "存在-10.78 kW平均偏差和高功率区低估"),
73     "08b_feature_importance": (str(DATA_FILE), "fig08b_feature_importance + RidgePowerModel", "§ 5.2；图9", "标准化系数用于预测解释，不
    ⇨ 代表单变量因果效应"),

```

```

74 "09_voltage_power_response": (str(DATA_FILE), "fig09_voltage_response + RidgePowerModel", " § 5.2; 图11", "局部情景曲线，仅在历史电
    ↳ 压附近解释"),
75 "10_optimal_power": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig10_power", " § 5.4; 图13", "功率来自数据模型；可行
    ↳ 性来自未辨识灰箱情景"),
76 "11_optimal_voltage": (str(RESULTS / "optimal_controls_central_scenario.csv"), "grouped_controls(U_COLS)", " § 6; 图14", "最优电压是情
    ↳ 景试验起点，不是设备安全设定"),
77 "12_optimal_rapping_period": (str(RESULTS / "optimal_controls_central_scenario.csv"), "grouped_controls(T_COLS)", " § 6; 图16", "无实际
    ↳ 振打事件，周期结果依赖峰值先验"),
78 "12b_radar_controls": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig12b_control_profiles", " § 6; 图15", "控制剖面来自中
    ↳ 心灰箱情景，不是设备安全设定"),
79 "13_emission_decomposition": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig13_emission", " § 7.2; 图18", "全部排放值为情
    ↳ 景推演，不是实测"),
80 "14_rapping_peak_mechanism": (str(RESULTS / "condition_profiles.csv"), "fig14_peak", " § 4.2; 图4", "峰值指数1.35与尺度1.2属于工程先
    ↳ 验"),
81 "15_search_convergence": (str(RESULTS / "optimization_seed_stability.csv"), "fig15_seed_stability", " § 8.1; 图19", "随机种子稳定不等
    ↳ 价于全局最优证明"),
82 "16_sensitivity_distribution": (str(RESULTS / "structural_sensitivity.csv"), "fig16_structural_sensitivity", " § 8.2; 图20", "扫描范围
    ↳ 仍由显式情景网格决定"),
83 "17_sensitivity_heatmap": (str(RESULTS / "field_priority_ablation_summary.csv"), "fig17_field_ablation", " § 8.3; 图21", "消融仍依赖同
    ↳ 一灰箱结构与支持域"),
84 "18_condition_energy_penalty": (str(RESULTS / "question4_by_condition.csv"), "fig18_penalty", " § 7.1; 图17", "13.69%为单一种子中心情
    ↳ 景点估计"),
85 "05b_boundary_proximity": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig05b_boundary_proximity", " § 5.4; 图12", "经验支
    ↳ 持域不是设备安全边界，贴边解仍需现场复核"),
86 }
87
88 MANUSCRIPT_FIGURE = {
89     "01_outlet_data_diagnosis": 1,
90     "02_process_timeseries": 2,
91     "06_method_overview": 3,
92     "14_rapping_peak_mechanism": 4,
93     "03_cluster_selection": 5,
94     "04_condition_map": 6,
95     "05_condition_profile_heatmap": 7,
96     "07_power_prediction": 8,
97     "08b_feature_importance": 9,
98     "08_power_residuals": 10,
99     "09_voltage_power_response": 11,
100     "05b_boundary_proximity": 12,
101     "10_optimal_power": 13,
102     "11_optimal_voltage": 14,
103     "12b_radar_controls": 15,
104     "12_optimal_rapping_period": 16,
105     "18_condition_energy_penalty": 17,
106     "13_emission_decomposition": 18,
107     "15_search_convergence": 19,
108     "16_sensitivity_distribution": 20,
109     "17_sensitivity_heatmap": 21,
110 }
111
112
113 def _label_bars(ax, bars, fmt="{:.0f}", dy=3.0):
114     for bar in bars:
115         h = bar.get_height()
116         ax.text(bar.get_x() + bar.get_width() / 2, h + dy, fmt.format(h), ha="center", va="bottom", fontsize=7)
117
118
119 def fig01_outlet_diagnosis(df: pd.DataFrame) -> None:
120     valid = df["C_out_mgNm3"].dropna()
121     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.2), gridspec_kw={"width_ratios": [1.15, 1]})
122     ax = axes[0]
123     add_subpanel_label(ax, "a")

```

```

124 bins = np.linspace(valid.min() - 0.03, valid.max() + 0.03, 28)
125 ax.hist(valid, bins=bins, color=PALETTE[0], edgecolor="white")
126 ax.axvline(50, color=PALETTE[4], linestyle="--", label="记录上限 50")
127 ax.set_xlim(valid.min() - 0.05, valid.max() + 0.05)
128 ax.text(
129     0.03,
130     0.78,
131     "5与10 mg/Nm³目标均在图示范围外",
132     transform=ax.transAxes,
133     color=PALETTE[2],
134     fontsize=8,
135     bbox={"boxstyle": "round", "facecolor": "white", "edgecolor": LIGHT_GRAY},
136 )
137 ax.set_xlabel("出口粉尘浓度记录 (mg/Nm³)")
138 ax.set_ylabel("样本数")
139 ax.legend(frameon=False, loc="upper left")
140 finish_axes(ax)
141
142 ax = axes[1]
143 add_subpanel_label(ax, "b")
144 total = len(df)
145 categories = ["精确值", "50截顶", "缺失"]
146 values = [int(df["C_out_exact_flag"].sum()), int(df["C_out_cap50_flag"].sum()), int(df["C_out_missing_flag"].sum())]
147 bars = ax.bar(categories, values, color=[PALETTE[0], PALETTE[4], GRAY])
148 for bar, value in zip(bars, values):
149     ax.text(bar.get_x() + bar.get_width()/2, value + total*0.012, f"{value}\n({100*value/total:.1f}%)", ha="center", fontsize=8)
150 ax.set_ylabel("样本数")
151 ax.set_ylim(0, max(values) * 1.18)
152 ax.text(0.02, 0.97, "百分比以全部10080条样本为分母", transform=ax.transAxes, va="top", fontsize=7, color=GRAY)
153 finish_axes(ax)
154 fig.tight_layout(w_pad=2.0)
155 save_figure(fig, OUT, "01_outlet_data_diagnosis")
156
157
158 def fig02_timeseries(df: pd.DataFrame) -> None:
159     d = df.iloc[:1440].copy()
160     hours = np.arange(len(d)) / 60
161     fields = [
162         ("Temp_C", "入口温度 (°C)", PALETTE[4]),
163         ("C_in_gNm3", "入口浓度 (g/Nm³)", PALETTE[0]),
164         ("Q_Nm3h", "烟气流量 (Nm³/h)", PALETTE[2]),
165         ("P_total_kW", "总功率 (kW)", PALETTE[3]),
166     ]
167     fig, axes = plt.subplots(4, 1, figsize=(8.2, 5.5), sharex=True)
168     for ax, (col, label, color), tag in zip(axes, fields, ["a", "b", "c", "d"]):
169         add_subpanel_label(ax, tag, x=-0.08, y=1.02)
170         ax.plot(hours, d[col], color=color, linewidth=0.8)
171         ax.set_ylabel(label)
172         finish_axes(ax)
173     axes[-1].set_xlabel("首日运行时间 (h)")
174     axes[-1].set_xlim(0, 24)
175     fig.tight_layout(h_pad=0.5)
176     save_figure(fig, OUT, "02_process_timeseries")
177
178
179 def fig03_cluster_selection(cluster_eval: pd.DataFrame) -> None:
180     fig, axes = plt.subplots(1, 2, figsize=(7.7, 3.0))
181     ax = axes[0]
182     add_subpanel_label(ax, "a")
183     ax.plot(cluster_eval["k"], cluster_eval["silhouette"], "o-", color=PALETTE[0])
184     chosen = cluster_eval[cluster_eval["selected"]].iloc[0]
185     ax.scatter([chosen["k"]], [chosen["silhouette"]], s=65, color=PALETTE[4], zorder=3, label="选定 k=4")

```

```

186 ax.set_xlabel("聚类数 k")
187 ax.set_ylabel("轮廓系数")
188 ax.legend(frameon=False)
189 finish_axes(ax)
190 ax = axes[1]
191 add_subpanel_label(ax, "b")
192 ax.plot(cluster_eval["k"], cluster_eval["inertia"] / 1000, "s--", color=PALETTE[2], label="惯性/1000")
193 ax.set_xlabel("聚类数 k")
194 ax.set_ylabel("类内惯性 ( $\times 10^3$ )")
195 finish_axes(ax)
196 fig.tight_layout(w_pad=2.0)
197 save_figure(fig, OUT, "03_cluster_selection")
198
199
200 def fig04_condition_map(df: pd.DataFrame) -> None:
201     train = df[df["split"] == "train"]
202     fig, ax = plt.subplots(figsize=(6.7, 4.4))
203     for cluster, group in train.groupby("condition_cluster"):
204         g = group.iloc[:5]
205         sizes = 8 + 28 * (g["Q_Nm3h"] - train["Q_Nm3h"].min()) / (train["Q_Nm3h"].max() - train["Q_Nm3h"].min())
206         ax.scatter(g["C_in_gNm3"], g["Temp_C"], s=sizes, alpha=0.38, color=PALETTE[int(cluster)-1], label=COND_SHORT[int(cluster)].replace("\
↪ n", " "))
207     ax.set_xlabel("入口粉尘浓度 (g/Nm3)")
208     ax.set_ylabel("入口温度 (°C)")
209     ax.legend(frameon=False, ncol=2)
210     finish_axes(ax, grid_axis="both")
211     fig.tight_layout()
212     save_figure(fig, OUT, "04_condition_map")
213
214
215 def fig05_profile_heatmap(profiles: pd.DataFrame) -> None:
216     cols = ["Temp_C", "C_in_gNm3", "Q_Nm3h", "dust_load_kg_h", "historical_power_kw"]
217     labels = ["温度", "入口浓度", "烟气流量", "粉尘负荷", "历史总功率"]
218     values = profiles[cols].to_numpy(float)
219     z = (values - values.mean(axis=0)) / values.std(axis=0)
220     fig, ax = plt.subplots(figsize=(7.2, 3.2))
221     im = ax.imshow(z, cmap="RdBu_r", vmin=-1.7, vmax=1.7, aspect="auto")
222     ax.set_xticks(range(len(labels)), labels)
223     ax.set_yticks(range(4), [COND_SHORT[i] for i in range(1, 5)])
224     for i in range(z.shape[0]):
225         for j in range(z.shape[1]):
226             ax.text(j, i, f"{z[i,j]:.2f}", ha="center", va="center", fontsize=8,
227                    color="white" if abs(z[i,j]) > 0.85 else "black")
228     cbar = fig.colorbar(im, ax=ax, fraction=0.032, pad=0.03)
229     cbar.set_label("列内标准分数")
230     fig.tight_layout()
231     save_figure(fig, OUT, "05_condition_profile_heatmap")
232
233
234 def fig05b_boundary_proximity(optimum: pd.DataFrame) -> None:
235     """Audit how close each recommended point lies to its empirical support limit."""
236     ordered = optimum.sort_values(["condition_cluster", "limit_mgNm3"], ascending=[True, False]).copy()
237     ordered["ratio_pct"] = 100.0 * ordered["mahalanobis_d2"] / ordered["support_threshold_d2"]
238     labels = [f"工况{int(c)}\n{int(lim)} mg" for c, lim in zip(ordered["condition_cluster"], ordered["limit_mgNm3"])]
239     colors = ["#D55E00" if ratio > 90 else PALETTE[int(c)-1]
240              for c, ratio in zip(ordered["condition_cluster"], ordered["ratio_pct"])]
241
242     fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.6), gridspec_kw={"width_ratios": [1.05, 1.2]})
243     add_subpanel_label(axes[0], "a")
244     x = np.arange(len(ordered))
245     axes[0].vlines(x, 0, ordered["ratio_pct"], color=colors, linewidth=2.2, alpha=0.8)
246     axes[0].scatter(x, ordered["ratio_pct"], color=colors, s=45, edgecolor="black", linewidth=0.5, zorder=3)

```



```

247 axes[0].axhline(90, color=PALETTE[1], linestyle=":", linewidth=1.2, label="90%贴近阈值")
248 axes[0].axhline(100, color=PALETTE[4], linestyle="--", linewidth=1.2, label="经验边界")
249 axes[0].set_xticks(x, labels)
250 axes[0].set_ylabel("边界相对距离  $d_M^2/q_{\{0.975\}}$  (%)")
251 axes[0].set_ylim(0, 108)
252 axes[0].legend(frameon=False, loc="lower left", fontsize=7.5)
253 finish_axes(axes[0], grid_axis="y")
254
255 add_subpanel_label(axes[1], "b")
256 y = np.arange(len(ordered))[:-1]
257 axes[1].barh(y, ordered["support_threshold_d2"], color=LIGHT_GRAY, height=0.58, label="经验97.5%边界")
258 axes[1].barh(y, ordered["mahalanobis_d2"], color=colors, height=0.38, label="推荐解距离")
259 axes[1].set_yticks(y, labels)
260 axes[1].set_xlabel("马氏距离平方")
261 axes[1].legend(frameon=False, fontsize=7.5)
262 finish_axes(axes[1], grid_axis="x")
263
264 fig.tight_layout(w_pad=1.8)
265 save_figure(fig, OUT, "05b_boundary_proximity")
266
267
268 def fig06_method_overview() -> None:
269     fig, ax = plt.subplots(figsize=(9.0, 3.7))
270     ax.set_xlim(0, 10)
271     ax.set_ylim(0, 4.5)
272     ax.axis("off")
273     stages = [
274         (0.25, 1.2, 1.55, 2.1, "原始数据", "过程量、总功率\n出口浓度记录"),
275         (2.1, 1.2, 1.65, 2.1, "数据审计", "截顶诊断\n时间切分与支持域"),
276         (4.05, 2.35, 1.75, 1.35, "数据驱动层", "工况聚类\n岭回归功率模型"),
277         (4.05, 0.35, 1.75, 1.35, "灰箱情景层", "捕集指数\n振打峰值代理"),
278         (6.15, 1.2, 1.65, 2.1, "约束优化", "5/10 mg情景约束\n历史支持域搜索"),
279         (8.15, 1.2, 1.55, 2.1, "输出", "分工况控制策略\n功率代价与敏感性"),
280     ]
281     fills = ["#E8F1F8", "#F2F4F7", "#E8F5F0", "#FFF2E5", "#F3ECF7", "#E8F1F8"]
282     for (x, y, w, h, title, body), fill in zip(stages, fills):
283         box = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.04,rounding_size=0.08", facecolor=fill, edgecolor="#4B5563", linewidth=1.0)
284         ax.add_patch(box)
285         ax.text(x+w/2, y+h*0.68, title, ha="center", va="center", fontsize=10, fontweight="bold")
286         ax.text(x+w/2, y+h*0.34, body, ha="center", va="center", fontsize=8.5, linespacing=1.45)
287     arrows = [
288         ((1.82, 2.25), (2.08, 2.25), "清洗数据"),
289         ((3.77, 2.52), (4.03, 2.85), "可辨识部分"),
290         ((3.77, 1.98), (4.03, 1.05), "不可辨识部分"),
291         ((5.82, 3.0), (6.13, 2.65), "功率预测"),
292         ((5.82, 1.05), (6.13, 1.75), "排放约束"),
293         ((7.82, 2.25), (8.13, 2.25), "最优解"),
294     ]
295     for start, end, label in arrows:
296         ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=10, color=GRAY, linewidth=1.1))
297         ax.text((start[0]+end[0])/2, (start[1]+end[1])/2+0.18, label, ha="center", fontsize=7, color=GRAY)
298     ax.text(4.93, 4.12, "实测证据", ha="center", color=PALETTE[2], fontsize=8, fontweight="bold")
299     ax.text(4.93, 0.04, "显式先验 (不冒充实测辨识)", ha="center", color=PALETTE[4], fontsize=8, fontweight="bold")
300     fig.tight_layout()
301     save_figure(fig, OUT, "06_method_overview")
302
303
304 def fit_power_model(df: pd.DataFrame) -> RidgePowerModel:
305     train = df[df["split"] == "train"]
306     metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
307     model = RidgePowerModel(alpha=float(metrics["alpha_selected_by_blocked_rolling_validation"]))
308     model.fit(train, train["P_total_kW"].to_numpy(float))

```

```

309     return model
310
311
312 def fig07_power_prediction(df: pd.DataFrame, model: RidgePowerModel, baselines: pd.DataFrame) -> None:
313     test = df[df["split"] == "test"]
314     actual = test["P_total_kw"].to_numpy(float)
315     pred = model.predict(test)
316     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5))
317     ax = axes[0]
318     add_subpanel_label(ax, "a")
319     ax.scatter(actual, pred, s=10, alpha=0.42, color=PALETTE[0], edgecolors="none")
320     lo, hi = min(actual.min(), pred.min()), max(actual.max(), pred.max())
321     ax.plot([lo, hi], [lo, hi], "--", color=PALETTE[4], label="理想预测")
322     ax.text(0.04, 0.94, "$R^2=0.957$\nRMSE=14.76 kW", transform=ax.transAxes, va="top",
323            bbox={"boxstyle": "round", "facecolor": "white", "edgecolor": "LIGHT_GRAY"})
324     ax.set_xlabel("实际总功率 (kW)")
325     ax.set_ylabel("预测总功率 (kW)")
326     ax.legend(frameon=False, loc="lower right")
327     finish_axes(ax, grid_axis="both")
328
329     ax = axes[1]
330     add_subpanel_label(ax, "b")
331     labels = ["均值基线", "线性岭", "二次岭"]
332     model_order = ["mean_predictor", "linear_ridge", "quadratic_ridge"]
333     x = np.arange(3)
334     val = baselines[baselines["split"] == "validation"].set_index("model").loc[model_order, "rmse_kw"]
335     tst = baselines[baselines["split"] == "retrospective_test"].set_index("model").loc[model_order, "rmse_kw"]
336     ax.bar(x-0.18, val, 0.36, color=PALETTE[5], label="验证集")
337     ax.bar(x+0.18, tst, 0.36, color=PALETTE[0], label="回顾性测试")
338     ax.set_xticks(x, labels)
339     ax.set_ylabel("RMSE (kW)")
340     ax.set_yscale("log")
341     ax.text(0.03, 0.96, "纵轴为对数刻度", transform=ax.transAxes, va="top", fontsize=7, color=GRAY)
342     ax.legend(frameon=False)
343     finish_axes(ax)
344     fig.tight_layout()
345     save_figure(fig, OUT, "07_power_prediction")
346
347
348 def fig08_residuals(df: pd.DataFrame, model: RidgePowerModel) -> None:
349     test = df[df["split"] == "test"]
350     actual = test["P_total_kw"].to_numpy(float)
351     pred = model.predict(test)
352     res = pred - actual
353     fig, axes = plt.subplots(1, 2, figsize=(7.7, 3.1))
354     add_subpanel_label(axes[0], "a")
355     axes[0].scatter(pred, res, s=9, alpha=0.38, color=PALETTE[0], edgecolors="none")
356     axes[0].axhline(0, color=PALETTE[4], linestyle="--")
357     axes[0].set_xlabel("预测总功率 (kW)")
358     axes[0].set_ylabel("残差: 预测-实际 (kW)")
359     finish_axes(axes[0], grid_axis="both")
360     add_subpanel_label(axes[1], "b")
361     axes[1].hist(res, bins=28, color=PALETTE[2], edgecolor="white")
362     metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
363     guard = float(metrics["validation_abs_error_q90_kw"])
364     axes[1].axvline(res.mean(), color=PALETTE[4], linestyle="--", label=f"测试均值 {res.mean():.2f} kW")
365     axes[1].axvline(guard, color=PALETTE[0], linestyle=":", label=f"验证|误差|P90 {guard:.2f} kW")
366     axes[1].set_xlabel("残差 (kW)")
367     axes[1].set_ylabel("样本数")
368     axes[1].legend(frameon=False)
369     finish_axes(axes[1])
370     fig.tight_layout(w_pad=2.0)

```

```

371     save_figure(fig, OUT, "08_power_residuals")
372
373
374 def fig08b_feature_importance(model: RidgePowerModel) -> None:
375     """Plot standardized ridge coefficients without giving them a causal meaning."""
376     names = [
377         "$U_1$", "$U_2$", "$U_3$", "$U_4$",
378         "$U_1^2$", "$U_1U_2$", "$U_1U_3$", "$U_1U_4$",
379         "$U_2^2$", "$U_2U_3$", "$U_2U_4$", "$U_3^2$", "$U_3U_4$", "$U_4^2$",
380         "$T_1$", "$T_2$", "$T_3$", "$T_4$", "入口温度 $H$", "入口浓度 $C_{in}$", "烟气流量 $Q$",
381     ]
382     coef = np.asarray(model.beta[1:], dtype=float)
383     order = np.argsort(np.abs(coef))
384     values = coef[order]
385     labels = [names[i] for i in order]
386     colors = [PALETTE[0] if value >= 0 else PALETTE[4] for value in values]
387
388     fig, ax = plt.subplots(figsize=(7.5, 4.2))
389     bars = ax.barh(np.arange(len(values)), values, color=colors, alpha=0.86, height=0.62)
390     for bar, value in zip(bars, values):
391         ax.text(value + (1.0 if value >= 0 else -1.0), bar.get_y() + bar.get_height()/2,
392             f"{value:+.1f}", va="center", ha="left" if value >= 0 else "right", fontsize=7)
393     ax.set_yticks(np.arange(len(values)), labels)
394     ax.set_xlabel("标准化岭回归系数 (kW/标准差)")
395     ax.axvline(0, color=GRAY, linestyle="--", linewidth=0.8)
396     finish_axes(ax, grid_axis="x")
397     fig.tight_layout()
398     save_figure(fig, OUT, "08b_feature_importance")
399
400
401 def fig09_voltage_response(df: pd.DataFrame, profiles: pd.DataFrame, model: RidgePowerModel) -> None:
402     train = df[df["split"] == "train"]
403     fig, ax = plt.subplots(figsize=(6.4, 4.0))
404     for _, p in profiles.iterrows():
405         c = int(p["condition_cluster"])
406         group = train[train["condition_cluster"] == c]
407         scale = np.linspace(0.92, 1.08, 80)
408         rows = []
409         u0 = group[U_COLS].median().to_numpy(float)
410         t0 = group[T_COLS].median().to_numpy(float)
411         for s in scale:
412             row = {col: val for col, val in zip(U_COLS, u0*s)}
413             row.update({col: val for col, val in zip(T_COLS, t0)})
414             row.update({"Temp_C": p["Temp_C"], "C_in_gNm3": p["C_in_gNm3"], "Q_Nm3h": p["Q_Nm3h"]})
415             rows.append(row)
416         y = model.predict(pd.DataFrame(rows))
417         ax.plot(100*(scale-1), y, color=PALETTE[c-1], marker=["o", "s", "A", "D"][c-1], markevery=16,
418             label=COND_SHORT[c].replace("\n", " "))
419     ax.set_xlabel("四电场电压同步变化 (%)")
420     ax.set_ylabel("模型预测总功率 (kW)")
421     ax.axvline(0, color=GRAY, linewidth=0.8)
422     ax.legend(frameon=False, ncol=2)
423     finish_axes(ax, grid_axis="both")
424     fig.tight_layout()
425     save_figure(fig, OUT, "09_voltage_power_response")
426
427
428 def fig10_power(optimum: pd.DataFrame, q4: pd.DataFrame) -> None:
429     p = optimum.pivot(index="condition_cluster", columns="Limit_mgNm3", values="predicted_power_kw")
430     x = np.arange(4)
431     fig, ax = plt.subplots(figsize=(7.2, 4.0))
432     b1 = ax.bar(x-0.18, p[10.0], 0.36, color=PALETTE[5], label="10 mg/Nm³")

```

```

443     b2 = ax.bar(x+0.18, p[5.0], 0.36, color=PALETTE[4], label="5 mg/Nm³")
444     _label_bars(ax, b1); _label_bars(ax, b2)
445     ax.set_xticks(x, [COND_SHORT[i] for i in range(1,5)])
446     ax.set_ylabel("情景最优功率 (kw)")
447     ax.set_ylim(0, max(p.max())*1.15)
448     ax.legend(frameon=False, ncol=2)
449     finish_axes(ax)
450     fig.tight_layout()
451     save_figure(fig, OUT, "10_optimal_power")
452
453
454
455 def grouped_controls(optimum: pd.DataFrame, cols: List[str], stem: str, ylabel: str) -> None:
456     fig, axes = plt.subplots(2, 2, figsize=(8.2, 5.8), sharey=True)
457     for ax, cluster, tag in zip(axes.flat, range(1,5), ["a", "b", "c", "d"]):
458         add_subpanel_label(ax, tag)
459         sub = optimum[optimum["condition_cluster"] == cluster].set_index("Limit_mgNm3")
460         x = np.arange(4)
461         ax.bar(x-0.18, sub.loc[10.0, cols], 0.36, color=PALETTE[5], label="10 mg/Nm³")
462         ax.bar(x+0.18, sub.loc[5.0, cols], 0.36, color=PALETTE[4], label="5 mg/Nm³")
463         ax.set_xticks(x, [c.split("_")[0] for c in cols])
464         ax.set_title(COND_SHORT[cluster].replace("\n", " "), fontsize=9)
465         finish_axes(ax)
466     axes[0,0].set_ylabel(ylabel); axes[1,0].set_ylabel(ylabel)
467     axes[0,1].legend(frameon=False, ncol=2, loc="upper right", bbox_to_anchor=(1.0, 1.18))
468     fig.tight_layout(h_pad=1.8)
469     save_figure(fig, OUT, stem)
470
471
472
473 def fig12b_control_profiles(optimum: pd.DataFrame) -> None:
474     """Compare voltage and rapping profiles for representative low/high-load conditions."""
475     fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.6))
476     x = np.arange(4)
477     fields = ["电场1", "电场2", "电场3", "电场4"]
478     styles = {
479         (1, 10.0): ("o--", PALETTE[0], "工况1, 10 mg"),
480         (1, 5.0): ("s-", PALETTE[0], "工况1, 5 mg"),
481         (3, 10.0): ("^--", PALETTE[2], "工况3, 10 mg"),
482         (3, 5.0): ("D-", PALETTE[2], "工况3, 5 mg"),
483     }
484     for ax, cols, ylabel, tag in [
485         (axes[0], U_COLS, "中心情景电压 (kV)", "a"),
486         (axes[1], T_COLS, "中心情景振打周期 (s)", "b"),
487     ]:
488         add_subpanel_label(ax, tag)
489         for (cluster, limit), (style, color, label) in styles.items():
490             row = optimum[(optimum["condition_cluster"] == cluster) & (optimum["Limit_mgNm3"] == limit)].iloc[0]
491             ax.plot(x, row[cols].to_numpy(float), style, color=color, label=label, linewidth=1.6, markersize=5)
492             ax.set_xticks(x, fields)
493             ax.set_ylabel(ylabel)
494             ax.legend(frameon=False, fontsize=7.2)
495             finish_axes(ax, grid_axis="both")
496     fig.tight_layout(w_pad=1.8)
497     save_figure(fig, OUT, "12b_radar_controls")
498
499
500
501 def fig13_emission(optimum: pd.DataFrame) -> None:
502     x = np.arange(8)
503     ordered = optimum.sort_values(["Limit_mgNm3", "condition_cluster"], ascending=[False, True])
504     base = ordered["scenario_base_emission_mgNm3"].to_numpy()
505     peak = ordered["scenario_peak_excess_mgNm3"].to_numpy()
506     colors = [PALETTE[0]]*4 + [PALETTE[4]]*4
507     fig, ax = plt.subplots(figsize=(8.0, 4.0))

```

```

495 ax.bar(x, base, color=colors, alpha=0.82, label="连续排放基值")
496 ax.bar(x, peak, bottom=base, color=colors, alpha=0.35, edgecolor=colors, label="振打峰值增量")
497 ax.hlines(10, -0.5, 3.5, colors=GRAY, linestyle="--", linewidth=1.0)
498 ax.hlines(5, 3.5, 7.5, colors=GRAY, linestyle="--", linewidth=1.0)
499 ax.text(3.45, 10.15, "10 mg/Nm³", ha="right", fontsize=8, color=GRAY)
500 ax.text(7.45, 5.15, "5 mg/Nm³", ha="right", fontsize=8, color=GRAY)
501 ax.set_xticks(x, [f"工况{i}" for i in range(1,5)]*2)
502 ax.set_ylabel("峰值总排放情景值 (mg/Nm³)")
503 ax.legend(frameon=False, ncol=2)
504 finish_axes(ax)
505 fig.tight_layout()
506 save_figure(fig, OUT, "13_emission_decomposition")
507
508
509 def fig14_peak(profiles: pd.DataFrame) -> None:
510     ratio = np.linspace(0.75, 1.25, 200)
511     fig, ax = plt.subplots(figsize=(6.4, 4.0))
512     for cluster in [1,3,4]:
513         p = profiles[profiles["condition_cluster"] == cluster].iloc[0]
514         peak = 1.2 * float(p["load_ratio_to_train_median"])**0.8 * ratio**1.35
515         ax.plot(100*(ratio-1), peak, color=PALETTE[cluster-1], marker=["o", "s", "A", "D"][cluster-1], markevery=35,
516               label=COND_SHORT[cluster].replace("\n", " "))
517     ax.axvline(0, color=GRAY, linewidth=0.8)
518     ax.set_xlabel("振打周期相对参考最优周期变化 (%)")
519     ax.set_ylabel("单次再飞扬峰值增量代理 (mg/Nm³)")
520     ax.legend(frameon=False)
521     finish_axes(ax, grid_axis="both")
522     fig.tight_layout()
523     save_figure(fig, OUT, "14_rapping_peak_mechanism")
524
525
526 def fig15_seed_stability(seed_data: pd.DataFrame) -> None:
527     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.4))
528     ax = axes[0]
529     add_subpanel_label(ax, "a")
530     ordered = seed_data.assign(key=seed_data["condition_cluster"].astype(str)+"-"+seed_data["limit_mgNm3"].astype(int).astype(str))
531     keys = [f"{c}-{int(limit)}" for c in range(1,5) for limit in (10.0,5.0)]
532     for j, key in enumerate(keys):
533         g = ordered[ordered["key"] == key]
534         median = g["predicted_power_kw"].median()
535         rel = 100*(g["predicted_power_kw"]/median-1)
536         jitter = np.linspace(-0.12,0.12,len(g))
537         ax.scatter(np.full(len(g),j)+jitter, rel, color=PALETTE[j%4], marker="o" if key.endswith("-10") else "s", s=24)
538         ax.vlines(j, rel.min(), rel.max(), color=LIGHT_GRAY, linewidth=2, zorder=0)
539     ax.axhline(0, color=GRAY, linewidth=0.8)
540     ax.set_xticks(range(8), [f"工况{c}\n{limit} mg" for c in range(1,5) for limit in (10,5)])
541     ax.set_ylabel("相对五种子中位数偏差 (%)")
542     ax.set_xlabel("工况—限值组合")
543     finish_axes(ax, grid_axis="both")
544
545     ax = axes[1]
546     add_subpanel_label(ax, "b")
547     groups = [seed_data[seed_data["limit_mgNm3"]==limit]["local_refinement_improvement_pct"] for limit in (10.0,5.0)]
548     boxes = ax.boxplot(groups, tick_labels=["10 mg/Nm³", "5 mg/Nm³"], patch_artist=True, widths=0.55)
549     for box, color in zip(boxes["boxes"], [PALETTE[5], PALETTE[4]]):
550         box.set_facecolor(color); box.set_alpha(0.75)
551     ax.scatter(np.repeat([1,2], [len(groups[0]),len(groups[1])]), np.concatenate(groups), s=10, color=GRAY, alpha=0.45)
552     ax.set_ylabel("局部细化相对初始库改善 (%)")
553     ax.set_xlabel("排放限值")
554     finish_axes(ax)
555     fig.tight_layout(w_pad=2.1)
556     save_figure(fig, OUT, "15_search_convergence")

```

```

557
558
559 def fig16_structural_sensitivity(structural: pd.DataFrame) -> None:
560     valid = structural[structural["all_conditions_feasible"] == True]
561     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5), sharey=True)
562     ax = axes[0]
563     add_subpanel_label(ax, "a")
564     scales = sorted(valid["outlet_scale"].unique())
565     data = [valid[valid["outlet_scale"]==x]["weighted_power_increase_pct"] for x in scales]
566     boxes = ax.boxplot(data, tick_labels=[f"x:{.2f}" for x in scales], patch_artist=True, showfliers=False)
567     for i, box in enumerate(boxes["boxes"]):
568         box.set_facecolor(PALETTE[i%len(PALETTE)]); box.set_alpha(0.72)
569     ax.set_xlabel("出口记录缩放系数")
570     ax.set_ylabel("5相对10 mg/Nm³加权功率增幅 (%)")
571     finish_axes(ax)
572
573     ax = axes[1]
574     add_subpanel_label(ax, "b")
575     phases = sorted(valid["phase_scale"].unique())
576     profiles = ["equal", "weak_front", "central_front"]
577     styles = ["o-", "s--", "A."]
578     labels = ["等权", "弱前级", "中心前级"]
579     for profile, style, label, color in zip(profiles, styles, labels, [PALETTE[0], PALETTE[2], PALETTE[4]]):
580         medians = [valid[(valid["phase_scale"]==p)&(valid["alpha_profile"]==profile)]["weighted_power_increase_pct"].median() for p in phases
581         ↵ ]
582         ax.plot(phases, medians, style, color=color, label=label)
583     ax.set_xlabel("振打相位叠加尺度")
584     ax.legend(frameon=False)
585     finish_axes(ax, grid_axis="both")
586     fig.tight_layout(w_pad=2.0)
587     save_figure(fig, OUT, "16_sensitivity_distribution")
588
589 def fig17_field_ablation(ablation: pd.DataFrame) -> None:
590     profiles = ["equal", "weak_front", "central_front"]
591     labels = ["等权", "弱前级", "中心前级"]
592     cols = [f"weighted_delta_U{i}_kV" for i in range(1,5)]
593     d = ablation.set_index("alpha_profile").loc[profiles, cols]
594     fig, axes = plt.subplots(1, 2, figsize=(7.8, 3.3), gridspec_kw={"width_ratios": [1.45, 1]})
595     ax = axes[0]
596     add_subpanel_label(ax, "a")
597     im = ax.imshow(d.to_numpy(), cmap="YlGnBu", aspect="auto", vmin=0)
598     ax.set_xticks(range(4), [f"U{i}" for i in range(1,5)])
599     ax.set_yticks(range(3), labels)
600     ax.set_xlabel("电场")
601     ax.set_ylabel("去除贡献权重设定")
602     for i in range(3):
603         for j in range(4):
604             ax.text(j, i, f"{d.iloc[i,j]:.2f}", ha="center", va="center", color="white" if d.iloc[i,j]>5 else "black", fontsize=8)
605     cbar = fig.colorbar(im, ax=ax, fraction=0.05, pad=0.04)
606     cbar.set_label("10→5 mg时加权电压增量 (kV)")
607
608     ax = axes[1]
609     add_subpanel_label(ax, "b")
610     shares = 100*ablation.set_index("alpha_profile").loc[profiles, "front_two_share_of_positive_adjustment"]
611     bars = ax.bar(labels, shares, color=[PALETTE[0], PALETTE[2], PALETTE[4]])
612     _label_bars(ax, bars, fmt="{:.1f}%", dy=0.7)
613     ax.set_ylabel("前两电场占正向电压增量 (%)")
614     ax.set_ylim(0,100)
615     finish_axes(ax)
616     fig.tight_layout(w_pad=2.0)
617     save_figure(fig, OUT, "17_sensitivity_heatmap")

```

```

618
619
620 def fig18_penalty(q4: pd.DataFrame) -> None:
621     x = np.arange(4)
622     weighted_p10 = float(np.sum(q4["share"] * q4["power_10_kw"]) / q4["share"].sum())
623     weighted_p5 = float(np.sum(q4["share"] * q4["power_5_kw"]) / q4["share"].sum())
624     weighted = 100.0 * (weighted_p5 / weighted_p10 - 1.0)
625     fig, ax = plt.subplots(figsize=(6.4, 3.8))
626     bars = ax.bar(x, q4.sort_values("condition_cluster")["increase_pct"], color=PALETTE[:4])
627     _label_bars(ax, bars, fmt="{: .2f}%", dy=0.12)
628     ax.axhline(weighted, color=PALETTE[4], linestyle="--", label=f"工况加权平均 {weighted:.2f}%")
629     ax.set_xticks(x, [COND_SHORT[i] for i in range(1,5)])
630     ax.set_ylabel("情景预测功率增幅 (%)")
631     ax.set_ylim(0, max(q4["increase_pct"])*1.22)
632     ax.legend(frameon=False)
633     finish_axes(ax)
634     fig.tight_layout()
635     save_figure(fig, OUT, "18_condition_energy_penalty")
636
637
638 def write_manifest() -> None:
639     script_hash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
640     model_hash = hashlib.sha256((SRC_DIR / "scenario_model.py").read_bytes()).hexdigest()
641     rows = []
642     for stem, caption in CAPTIONS.items():
643         source_data, function_name, claims, limitations = TRACE_META[stem]
644         rows.append({"figure_id": stem.split("_")[0], "manuscript_figure": MANUSCRIPT_FIGURE[stem], "stem": stem, "caption": caption,
645                     "png": str((OUT/f"{stem}.png").resolve()), "pdf": str((OUT/f"{stem}.pdf").resolve()),
646                     "source_data": source_data,
647                     "transformation": f"paper_figures.py::{function_name}; plot_sha256={script_hash}; model_sha256={model_hash}",
648                     "supported_manuscript_claims": claims,
649                     "limitations": limitations})
650     pd.DataFrame(rows).to_csv(OUT / "figure_manifest.csv", index=False, encoding="utf-8-sig")
651     lines = ["# 论文图组与图注", "", "所有图均提供 PNG 预览和 PDF 矢量版本。", ""]
652     for row in sorted(rows, key=lambda x: x["manuscript_figure"]):
653         lines.extend([
654             f"## 正文图{row['manuscript_figure']} (文件编号{row['figure_id']}) {row['stem']}", "", row["caption"], "",
655             f"- 源数据: `{row['source_data']}`",
656             f"- 变换: `{row['transformation']}`",
657             f"- 支持的正文论断: {row['supported_manuscript_claims']}",
658             f"- 局限: {row['limitations']}",
659             f"- PNG: `{row['png']}`", f"- PDF: `{row['pdf']}`", ""])
660     (OUT / "图注清单.md").write_text("\n".join(lines), encoding="utf-8")
661
662
663 def main() -> None:
664     set_paper_style()
665     df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
666     profiles = pd.read_csv(RESULTS / "condition_profiles.csv")
667     cluster_eval = pd.read_csv(RESULTS / "cluster_selection_metrics.csv")
668     optimum = pd.read_csv(RESULTS / "optimal_controls_central_scenario.csv")
669     seed_data = pd.read_csv(RESULTS / "optimization_seed_stability.csv")
670     structural = pd.read_csv(RESULTS / "structural_sensitivity.csv")
671     ablation = pd.read_csv(RESULTS / "field_priority_ablation_summary.csv")
672     baselines = pd.read_csv(RESULTS / "power_model_baselines.csv")
673     q4 = pd.read_csv(RESULTS / "question4_by_condition.csv")
674     model = fit_power_model(df)
675
676     fig01_outlet_diagnosis(df)
677     fig02_timeseries(df)
678     fig03_cluster_selection(cluster_eval)
679     fig04_condition_map(df)

```

```

680 fig05_profile_heatmap(profiles)
681 fig05b_boundary_proximity(optimum)
682 fig06_method_overview()
683 fig07_power_prediction(df, model, baselines)
684 fig08b_feature_importance(model)
685 fig08_residuals(df, model)
686 fig09_voltage_response(df, profiles, model)
687 fig10_power(optimum, q4)
688 grouped_controls(optimum, U_COLS, "11_optimal_voltage", "最优电压 (kV)")
689 fig12b_control_profiles(optimum)
690 grouped_controls(optimum, T_COLS, "12_optimal_rapping_period", "最优振打周期 (s)")
691 fig13_emission(optimum)
692 fig14_peak(profiles)
693 fig15_seed_stability(seed_data)
694 fig16_structural_sensitivity(structural)
695 fig17_field_ablation(ablation)
696 fig18_penalty(q4)
697 write_manifest()
698 print(f"Generated {len(CAPTIONS)} figures in {OUT}")
699
700
701 if __name__ == "__main__":
702     main()

```

D.3 plot_utils.py

```

1  """Shared publication plotting style for the cement ESP paper."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8
9
10 PALETTE = ["#0072B2", "#E69F00", "#009E73", "#CC79A7", "#D55E00", "#56B4E9"]
11 GRAY = "#6B7280"
12 LIGHT_GRAY = "#D1D5DB"
13
14
15 def set_paper_style() -> None:
16     plt.rcParams.update(
17         {
18             "font.family": "sans-serif",
19             "font.sans-serif": [
20                 "Arial Unicode MS",
21                 "PingFang SC",
22                 "Heiti TC",
23                 "Noto Sans CJK SC",
24                 "DejaVu Sans",
25             ],
26             "axes.unicode_minus": False,
27             "font.size": 9.5,
28             "axes.labelsize": 9.5,
29             "axes.titlesize": 9.5,
30             "xtick.labelsize": 8.5,
31             "ytick.labelsize": 8.5,
32             "legend.fontsize": 8.5,
33             "axes.linewidth": 0.8,
34             "lines.linewidth": 1.6,
35             "grid.color": "#E5E7EB",

```



```

36         "grid.linewidth": 0.55,
37         "grid.alpha": 0.7,
38         "figure.dpi": 160,
39         "savefig.dpi": 300,
40         "savefig.bbox": "tight",
41         "pdf.fonttype": 42,
42         "ps.fonttype": 42,
43     }
44 )
45
46
47 def finish_axes(ax, grid_axis: str = "y") -> None:
48     if grid_axis != "none":
49         ax.grid(True, axis=grid_axis)
50     ax.set_axisbelow(True)
51     ax.spines["top"].set_visible(False)
52     ax.spines["right"].set_visible(False)
53
54
55 def add_subpanel_label(ax, label: str, x: float = -0.12, y: float = 1.05) -> None:
56     """Add a compact (a), (b), ... identifier to a multi-panel figure."""
57     ax.text(
58         x,
59         y,
60         f"({label})",
61         transform=ax.transAxes,
62         fontsize=10,
63         fontweight="bold",
64         va="bottom",
65         ha="right",
66     )
67
68
69 def save_figure(fig, out_dir: Path, stem: str) -> None:
70     out_dir.mkdir(parents=True, exist_ok=True)
71     fig.savefig(out_dir / f"{stem}.png")
72     fig.savefig(out_dir / f"{stem}.pdf")
73     plt.close(fig)

```